

반려인-시터 매칭 서비스

FOR-PETS

집사조일체

최길중 | 권지원 | 박영수 | 선경안 | 이지민

배포

<https://forpetscare.uk>



스파르타 내일배움캠프
최종 프로젝트

GitHub

<https://github.com/Housekeeper-for-pets>

✓ 목차

프로젝트 소개

주제 선정 배경
서비스 차별점
팀 소개
개발 일정

01

핵심 기능 구현

주요 기능
동시성 제어
성능 개선
AI 기능

03

설계 및 인프라

ERD
API 설계
AWS 아키텍처
개발 환경

02

검증 및 회고

테스트 결과
트러블슈팅
회고
Q&A

04

프로젝트 소개

주제 선정 배경

서비스 차별점

팀 소개

개발 일정

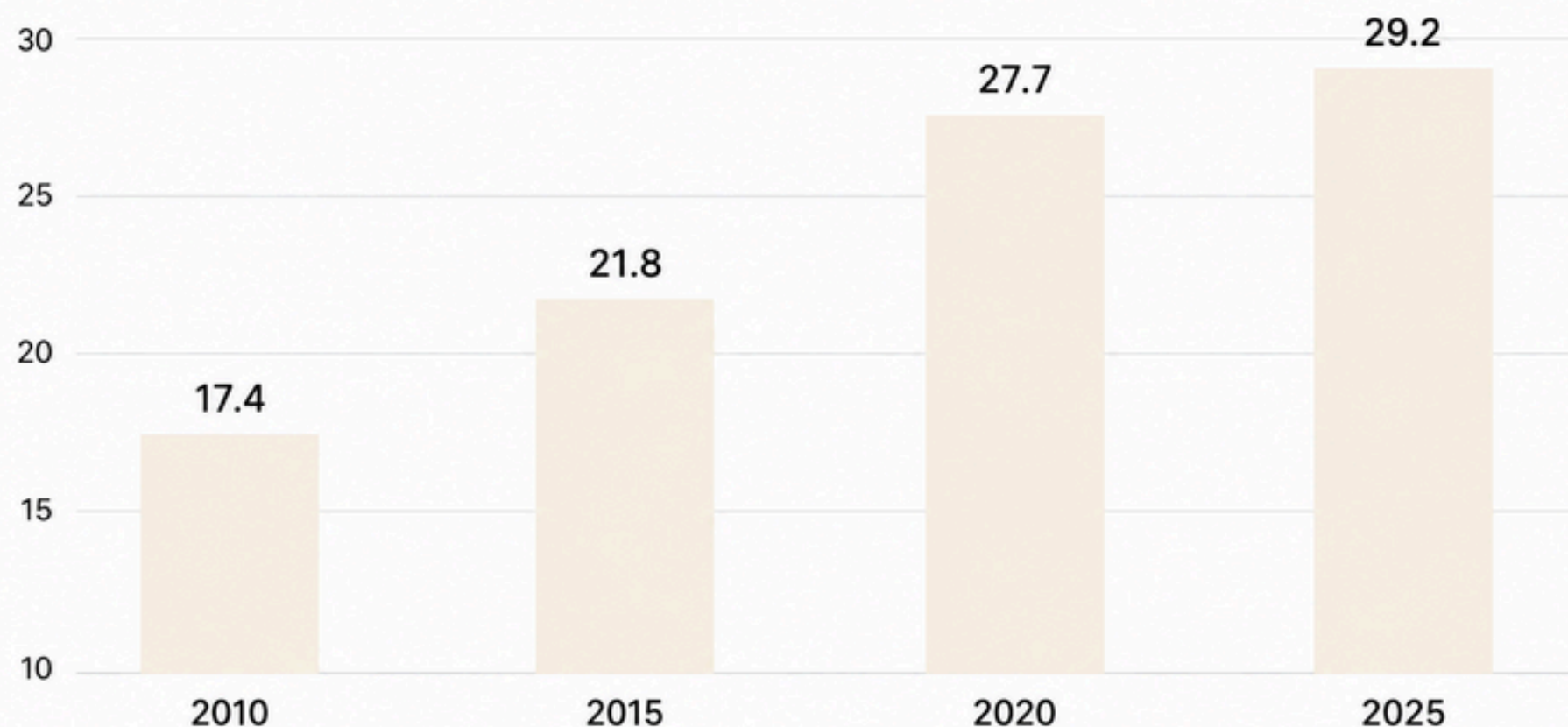


주제 선정 배경

2010 ~ 2025 국내 반려동물 양육비율

자료 : 농림축산식품부

(단위 : %)



2010년: 전화조사(2천명)

2015년: 전화조사/온라인 조사 병행 (3천명)

2020년: 온라인조사 (5천명)

2025년: 면접 조사 (3천가구) - 반려동물 양육 현황 조사 (국가 승인 통계)

국내 반려동물 양육 가구 비율 역대 최대 29.2%

- 1~2인 가구 및 맞벌이 가구 증가에 따른 반려동물 돌봄이 필요한 가구 확대
- 전문 펫시터의 돌봄 서비스 수요 증가
- 반려동물 시장 규모 증가
(2025년 3조 원 → 2027년 6조 원 전망)

출처: 한국 농촌 경제 연구원(KREI)

✓ 기존 서비스 분석 및 차별점

✓ 기존 서비스 분석

국내 펫시터 매칭 서비스 : **순방향 구조** 중심

- 보호자가 직접 돌봄 요청 → 후기 · 프로필 · 비용 비교 부담
- 예약 이후 소통 · 결제 · 정산 분절

✓ ForPets 차별점

역방향 매칭

시터 → 보호자
공고에 직접 제안
보호자의 검색 부담 해소

AI 기반 선택 보조

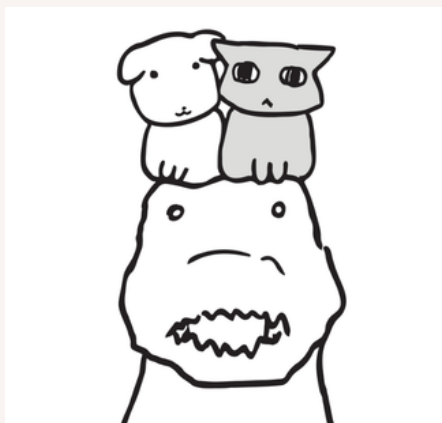
리뷰 요약 AI
시터 추천 AI
→ 비교 비용 절감

통합 예약 관리

예약 - 쿠폰 - 결제
정산 - 채팅 - 케어일지
단일 흐름



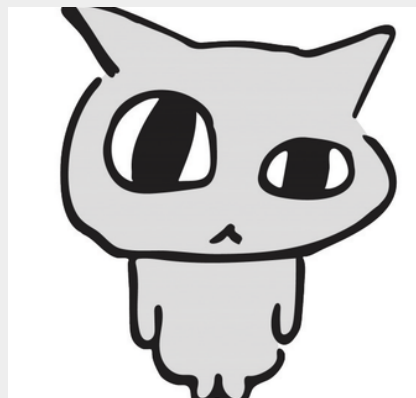
팀원 소개



최길중

리더
인증/인가, 채팅, 쿠폰 동시성

충땅



권지원

메인 비즈니스 로직
스케줄링, 동시성, 모니터링

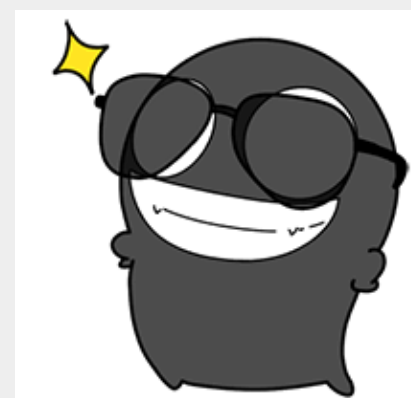
4랑하는
4조



박영수

AWS 기반 인프라 구축,
CI/CD 배포 환경 구성,
실시간 알림 시스템

불태웠다..
오얼



선경안

조회 성능개선(캐싱/인덱싱)
리뷰, 모니터링

함께라면



이지민

결제·정산 / AI 기능
프론트엔드

4릉흔드..
연두맘





ForPets 마일스톤



설계 및 인프라

ERD

API 설계

협업툴

AWS 아키텍처

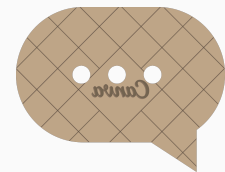
개발 환경

Git Workflow

Flowchart



디렉토리 구조



3 Layer architecture

도메인 중심 패키지 구조 → 응집도
명확한 레이어별 책임 분리

```

└─ monitoring/
  │ └─ grafana/dashboards/      # Grafana 대시보드 JSON
  │ └─ prometheus*.yaml        # Prometheus 환경별 설정
  │
└─ nginx/                      # 환경별 Nginx 설정
└─ k6/                         # 부하 테스트 스크립트
└─ Dockerfile
└─ docker-compose.yml
└─ docker-compose.prod.yml
└─ build.gradle
  
```

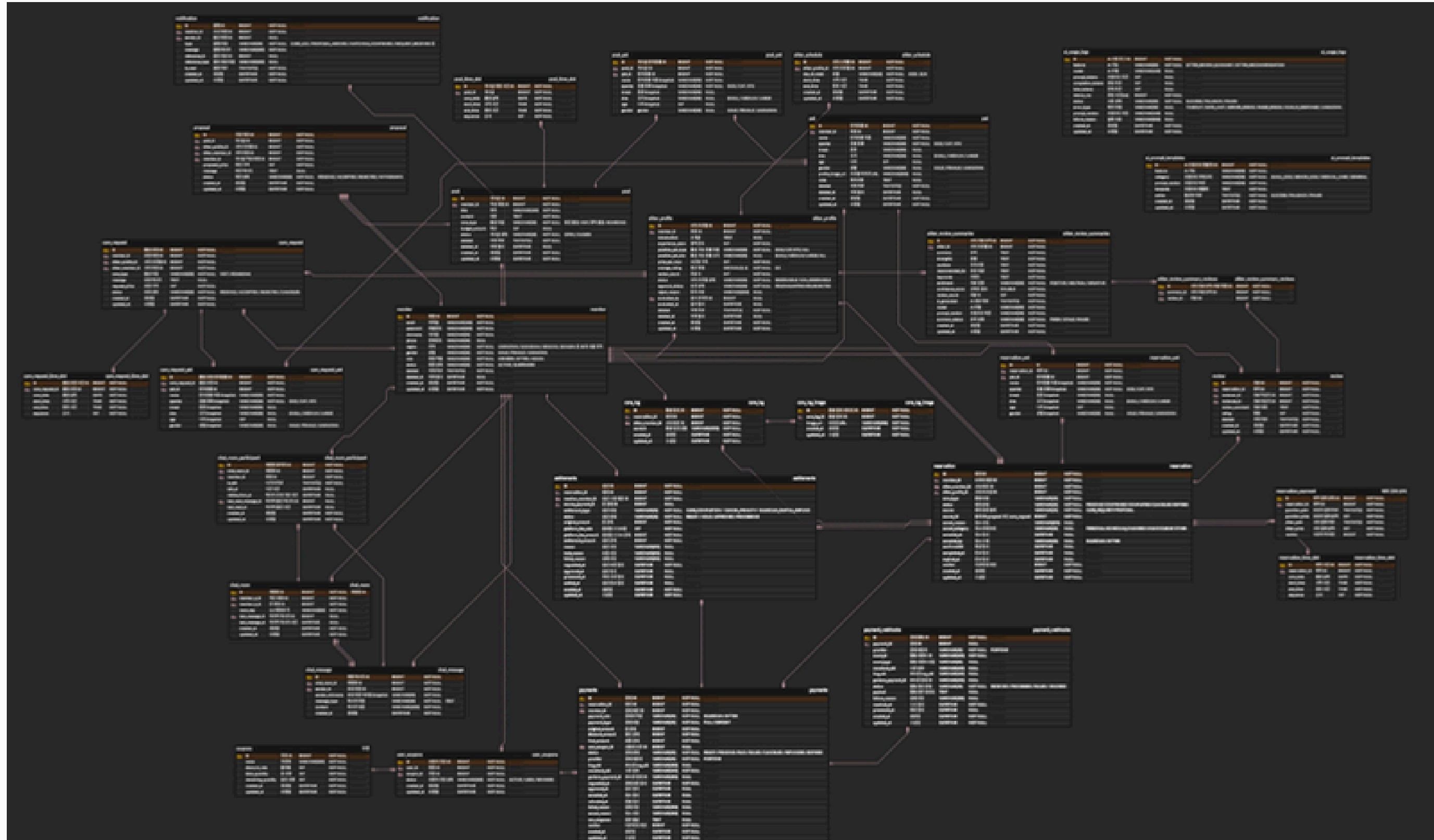
```

for-pets-project/
└─ src/main/java/com/forpets/
  │ └─ domain/
  │   │ └─ admin/              # 관리자 (예약·시터 관리)
  │   │ └─ ai/                 # AI (채팅·RAG 벡터 검색·리뷰 요약·사용량 로그)
  │   │ └─ auth/              # 인증 (JWT 발급·재발급)
  │   │ └─ carelog/           # 케어 일지
  │   │ └─ carerequest/       # 케어 요청
  │   │ └─ chat/              # 실시간 채팅 (WebSocket + Redis Stream)
  │   │ └─ coupon/            # 쿠폰 발급·사용
  │   │ └─ member/            # 회원
  │   │ └─ notification/      # 알림 (SSE + Kafka)
  │   │ └─ payment/           # 결제 (토스페이먼츠)
  │   │ └─ pet/               # 반려동물 등록·관리
  │   │ └─ post/              # 구인·구직 게시글
  │   │ └─ proposal/          # 시터 제안
  │   │ └─ reservation/      # 예약
  │   │ └─ review/            # 리뷰
  │   │ └─ settlement/        # 정산
  │   │ └─ sitter/            # 시터 프로필
  │   │
  │   └─ global/
  │     │ └─ config/           # Redis·Kafka·WebSocket·QueryDSL 설정
  │     │ └─ exception/       # 공통 예외 처리
  │     │ └─ security/         # Spring Security, JWT
  │     │ └─ scheduler/        # 만료 처리 스케줄러
  │     │ └─ aspect/           # 분산 락 AOP
  │     │ └─ monitoring/       # 실행 시간 추적 AOP
  │     │ └─ sse/              # SSE 연결 관리
  │
  └─
  
```


100



ERD

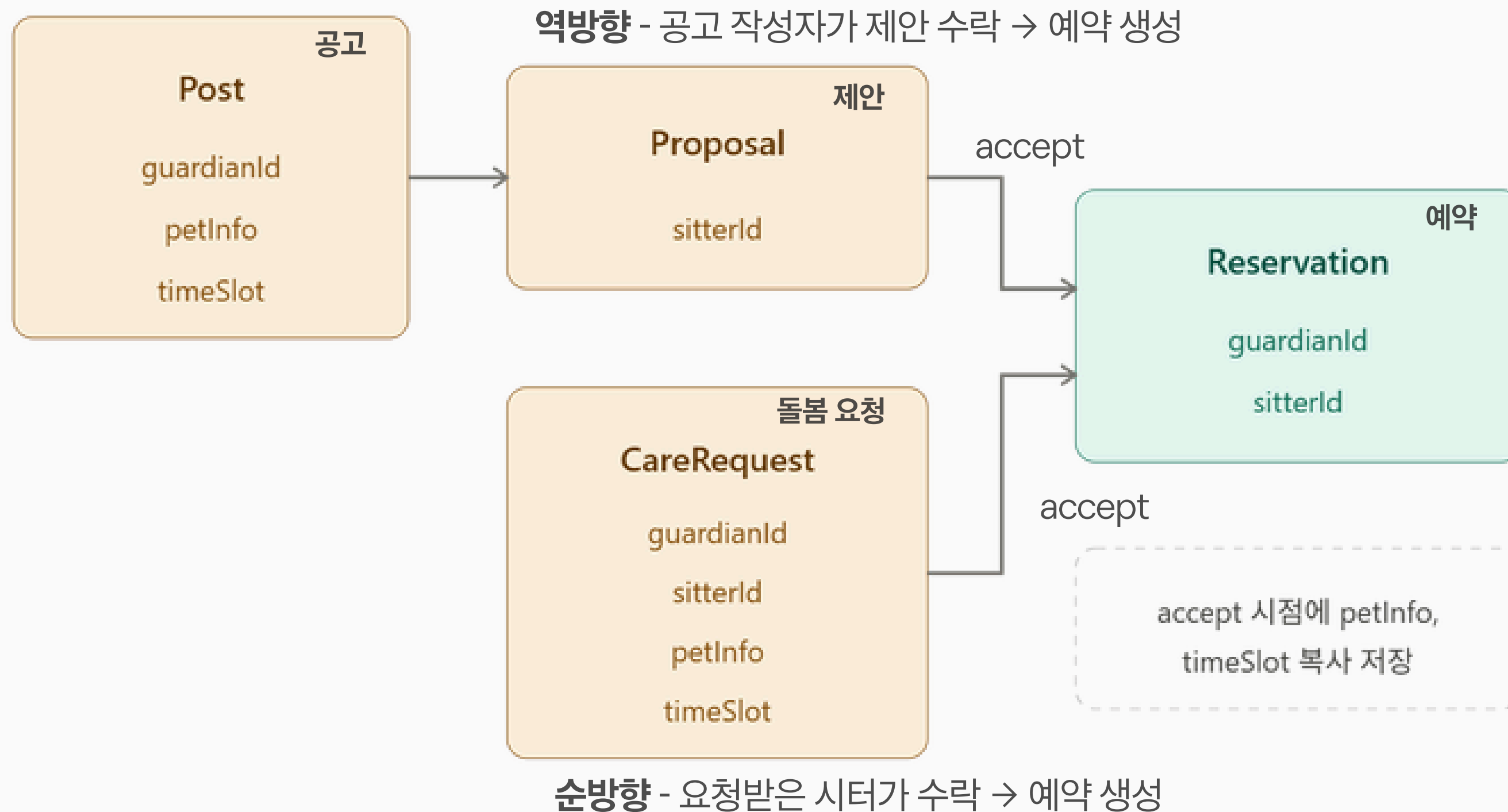


[illegible]



ERD

매칭 생성 부분 ERD 흐름



역방향 매칭 플로우





기술 스택

Backend

Java 21

Spring Boot 3.5.14

Spring Web

Spring Data JPA

Spring Security

QueryDSL

JWT

Lombok

Database

MySQL 8.4

H2

Vector DB

Qdrant 1.9.7

Cache / Lock

Redis 7

Redisson 3.27.2

Messaging

Kafka 3.7.0

Real-time

WebSocket

STOMP

AI

Spring AI

Gemini API

Infra / DevOps

Docker

Docker Compose

Nginx

AWS EC2

AWS ECR

GitHub Actions

Test / Monitoring

JUnit 5

k6

Prometheus

Grafana

Actuator

Micrometer

Collaboration

GitHub

Notion

IntelliJ IDEA

Postman

일정관리 - Jira

코드 첩삭 - Github

The screenshot shows a project management interface for a project named '집사조' (Jisajjo). The main view is a Gantt chart showing the timeline of various tasks. The tasks are listed on the left, and their durations are represented by horizontal bars on the right. The timeline spans from May 17 to June 9. The tasks are color-coded: yellow for planning/proposal, purple for development, blue for testing, and orange for deployment. The chart shows a complex schedule with many overlapping tasks and sub-tasks.

Task	Start Date	End Date	Color
PET-13 역방향 매칭 (Proposal)	May 17	May 20	Yellow
PET-14 예약	May 17	May 31	Yellow
PET-43 배포와 CI/CD	May 17	Jun 7	Purple
PET-22 데이터베이스	May 17	May 20	Purple
PET-23 결제	May 20	May 27	Blue
PET-25 Docs	May 25	May 28	Purple
PET-29 AI	May 25	Jun 9	Orange
PET-33 AI 서버 리뷰 요약 - LL...	May 27	May 30	Orange
PET-36 멀티턴 챗봇 + SSE 스...	May 27	May 30	Orange
PET-34 RAG 파이프라인 구축	May 27	May 30	Orange
PET-30 AI 서버 추천 챗봇(Too...	May 27	May 30	Orange
PET-37 코드 리팩토링	May 27	May 30	Orange
PET-32 AI 장애 격리	May 27	May 30	Orange
PET-38 AI 기능 통합 테스트...	May 27	May 30	Orange
PET-54 쿼몬	May 20	May 23	Purple
PET-55 채팅	May 20	May 23	Purple



협업툴

문서 - Notion

2. JWT / 인증 정책

- Access Token 만료 시간은 1시간이다.
- 로그아웃 시 Access Token을 Redis 블랙리스트에 등록한다.
키: jwt:blacklist:{token} / TTL: 토큰 잔여 만료 시간\ Refresh Token도 함께 무효화한다.
- 블랙리스트에 등록된 토큰은 JwtAuthFilter에서 차단한다.
- Refresh Token 만료 시간은 7일
- Refresh Token 저장 위치 : Redis
- Refresh Token 재발급 시 기존 토큰을 무효화한다. (Rotation 전략)

📌 지원 권 5월 15일
길종님 구현 추가 파트

3. 시터 프로필 정책

- MEMBER 가 시터 프로필을 작성하여 등록을 요청하면 ADMIN 이 approve api 를 통해 member.role = SITTER로 변경한다.
- 시터 프로필 삭제 시 member.role = MEMBER로 복원된다.
- 시터 프로필 재등록 시 다시 ADMIN 승인이 필요하고, 받은 리뷰 등은 복구된다.
- ADMIN은 시터 프로필을 등록할 수 없다. -> DB 에 직접 등록, 모든 권한 부여
- 회원 1명은 시터 프로필을 1개만 등록할 수 있다.
- 진행 중 예약이 있으면 시터 프로필을 삭제할 수 없다.
- 시터의 상태를 RESERVABLE / NON_RESERVABLE 로 구분
- 상태 전환:
 - 시터가 직접 토글 가능
 - 진행 중 예약(PENDING/CONFIRMED) 있어도 NON_RESERVABLE로 변경 가능
 - NON_RESERVABLE 상태에서는 신규 CareRequest/Proposal 불가

📌 지원 권 5월 15일
삭제 이후 UI 에서 검색 관련한 정책을 다시 정리

📌 지원 권 5월 15일
Member 쪽으로 정책 이동하거나 MVP 기준으로는 ADMIN 을 완전 제외하여 Sitter profile 을 등록하려고 하면 바로 Sitter 역할을 부여

4. 시터 주간 스케줄 정책

시터 스케줄 역할

공지 및 PR 알림 - Slack



박영수(Spring_3기) 오후 3:21

@here 금일 한 16시 쯤에 main배포 예정입니다.(nginx적용, 도메인 DNS 연결)
경안님처럼 prod 에 db 테이블 변경이 필요한 경우 저한테 쿼리랑 같이해서 내용 공유주시면 RDS에 반영 (그전에 PR 있으신분들은 미리끝내주세요! 아니면 배포이후에 부탁드립니다.)

cc. @선경안(Spring_3기) (편집됨)

넵! 3 🤔



GitHub 앱 오후 3:25

Pull request opened by giljung8228

#128 Feature/pet 215

- 쿠폰 단위, 통합 테스트 코드
- 인증 단위, 통합 테스트 코드
- 채팅 단위, 통합 테스트 코드

Reviewers

@박영수(Spring_3기)

Housekeeper-for-pets/for-pets-project | 6월 9일 | 봇이 추가한 GitHub

Comment

1개의 댓글 7일 전



GitHub 앱 오후 3:26

Pull request opened by jiiimni

#129 Feature/pet 34

Reviewers

@박영수(Spring_3기), @grown-tree, @권지 원(Spring_3기)

Assignees

@jiiimni



와이어프레임

홈

내 공고 4

시터 찾기

예약

반려동물

메시지

내 정보

조건을 적어두면 시터가 직접 제안을 보내드려요.

1 반려동물

2 일정·지역

3 돌봄 조건

4 공고 내용

어떤 친구를 맡길까요? · 10마리 이상

DOG 뽕치 · 말티즈

CAT 나비 · 꼬리만숏헤어

DOG 보리 · 시바견

+ 새 반려동물 등록

희망 날짜 *

2026-05-20 (수)

시작 시간

09:00

희망 지역 *

서울 강남구 역삼동

종료 시간

18:00

돌봄 유형 *

방문 돌봄
시터가 우리 집을 방문해요

위탁 돌봄
시터의 집에 맡겨요

희망 예산 (선택)

₩ 120,000

시간당 약 13,333원

제안 시 참고용으로 시터에게 전달해요

제목 *

5월 20일 강아지·고양이 종일 돌봄 부탁드려요

실시간 미리보기

모로 초 draft

5월 20일 강아지·고양이 종일 돌봄 부탁드려요

출장으로 인해 09시부터 18시까지 집을 비웁니다. 산책 2회, 급식 1회, 화장실 청소 부탁드립니다. 뽕치는 낯선 사람에게 낯지않 5분이면 적응합니다.

일정

5월 20일 (수) 09:00-18:00

지역

서울 강남구 역삼동

돌봄 유형

방문 돌봄

예산

120,000원

맡길 친구

DOG 뽕치 · 말티즈 · 3살

CAT 나비 · 꼬리만숏헤어 · 5살

잘 작성된 공고는 평균 23분 안에 첫 제안을 받아요

유형·시간·예산을 모두 채우면 매칭률이 높아집니다.

홈

내 공고 4

시터 찾기

예약

반려동물

메시지

내 정보

제안 비교

4명의 시터가 제안을 보냈어요. 신중하게 비교해보세요.

희망 예산 120,000원

희망 일일 5월 20일 (수) · 09:00-18:00

댕댕시터
후 서울 강남구

96% 매칭률

주요가 120,000원

경력 · 후기 5년 · ★ 4.9
후기 142개

가능 시간 5월 20일 (수) · 09:00 ~ 18:00

뽕치와 나비 모두 돌봐드릴 수 있습니다. 강아지와 고양이 함께 케어한 경험이 많아 두 친구 모두 편하게 지낼 수 있을 거예요. 산책 2회 포함하여 진행하겠습니다.

소형견 전문, 산책 가능, 급식

이 시터 채택하기

메시지, 거절

포포시터
후 서울 서초구

88% 매칭률

주요가 135,000원

경력 · 후기 7년 · ★ 4.8
후기 89개

가능 시간 5월 20일 (수) · 09:00 ~ 18:30

대형견과 고양이를 동시에 돌본 경험이 풍부합니다. 추가로 30분 더 머무르며 마무리해드릴 수 있어요. 케어 일지를 사진과 함께 보내드립니다.

대형견 가능, 고양이 전문, 급료초지

이 시터 채택하기

메시지, 거절

멍멍이모
후 서울 송파구

72% 매칭률

주요가 96,000원

예산 대비 -24,000원

경력 · 후기 3년 · ★ 4.7
후기 51개

가능 시간 5월 20일 (수) · 10:00 ~ 18:00

말티즈 케어 경험 있습니다. 나비(고양이)는 함께 케어가 어려워 잠시 케어 가능한 점 알려 부탁드립니다.

소형견 전문, 가격 합리

이 시터 채택하기

메시지, 거절

홈

내 공고 4

시터 찾기

예약

반려동물

메시지

내 정보

5월 20일 강아지·고양이 종일 돌봄 부탁드려요

일일 5월 20일 (수) · 09:00-18:00

지역 서울 강남구 역삼동

돌봄 유형 방문 돌봄

희망 예산 120,000원

종료 47명

공고 내용

출장으로 인해 09시부터 18시까지 집을 비웁니다. 산책 2회, 급식 1회, 화장실 청소 부탁드립니다. 뽕치는 낯선 사람에게 낯지않 5분이면 적응합니다.

맡길 친구

DOG 뽕치 · 말티즈 · 3살

CAT 나비 · 꼬리만숏헤어 · 5살

받은 제안 4건 새 제안 3

정렬: 매칭률

한눈에 비교

시터	매칭률	예산가	가능시간	경력	도움 시작	상세	
댕댕시터 후 서울 강남구 · ★ 4.9	96%	120,000원	09:00 ~ 18:00	5년	4분 전	상세 >	
포포시터 후 서울 서초구 · ★ 4.8	88%	135,000원	09:00 ~ 18:30	7년	12분 전	상세 >	
멍멍이모 후 서울 송파구 · ★ 4.7	72%	96,000원	-24,000	10:00 ~ 18:00	3년	21분 전	상세 >

공고 상태

상태

마감까지 0-7

조회수 47

받은 제안 42

제안 비교하기

공고 공유

제약 가이드

- 제한기간 보지 않고 경력·후기를 함께 보세요
- 제약 시 다른 제안은 자동 거절돼요
- 제약 후 24시간 내 예약을 확정해야 해요

홈

내 공고 4

시터 찾기

예약

반려동물

메시지

내 정보

댕댕시터님과 예약

1 예약 대기 05.14

2 예약 확정 -

3 케어 완료 -

4 후기 작성 -

시터 정보

댕댕시터
후 서울 강남구 · 시간당 15,000원

경력 5년 · ★ 4.9 (142)

소형견 전문, 산책 가능, 급식

메시지

케어 일정

날짜 5월 20일 (수)

시간 09:00 ~ 18:00
총 9시간

종류 방문 돌봄
서울 강남구 역삼동

맡길 친구

DOG 뽕치 · 말티즈 · 소형

CAT 나비 · 꼬리만숏헤어 · 소형

요율 88%

산책 2회 포함드립니다. 뽕치는 낯선 사람에게 낯지않 5분이면 적응해요.

결제 금액

제안가 (9시간) 120,000원

결제금 수수료 (5%) 6,000원

합인 126,000원

예약 확정하기

예약 취소

24시간 내 미확정 시 자동 환불됩니다.

상태 변화

05.14 09:30 역방문 제안 채택

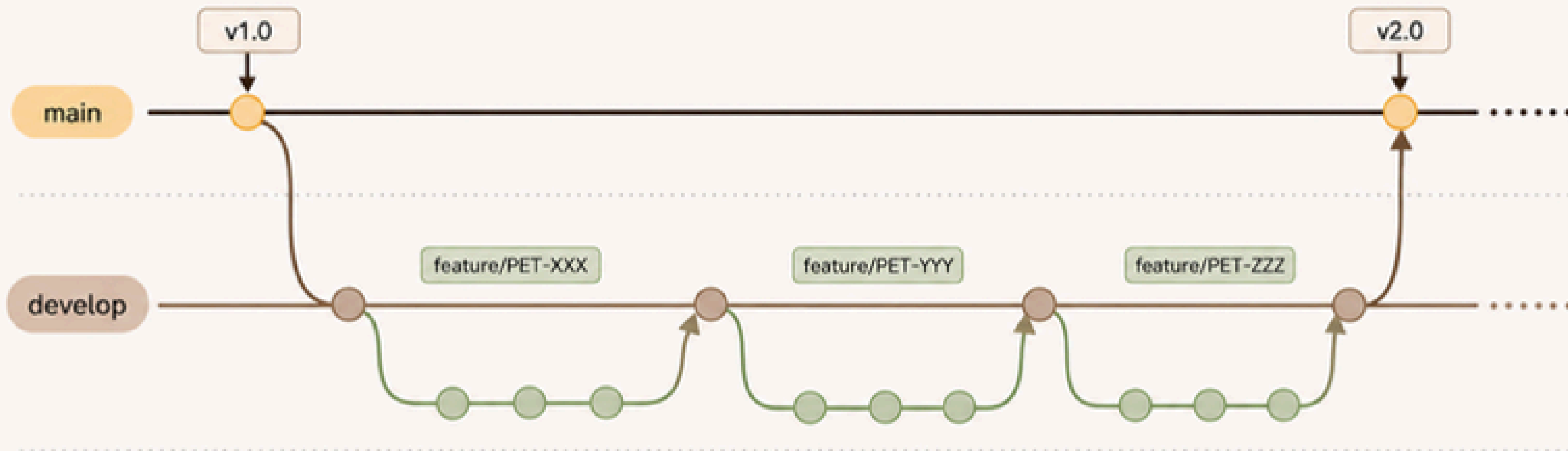
05.14 09:30 예약 생성 (PENDING)



Git Workflow

Git Workflow

● main
 ● develop
 ● feature/PET-{jira번호}



1. Branch 생성
feature/PET-{jira번호}
브랜치 생성 후 작업



2. PR & Merge
작업 완료 후
develop 브랜치로 PR



3. 통합 & 테스트
develop 브랜치에서
통합 및 테스트 진행



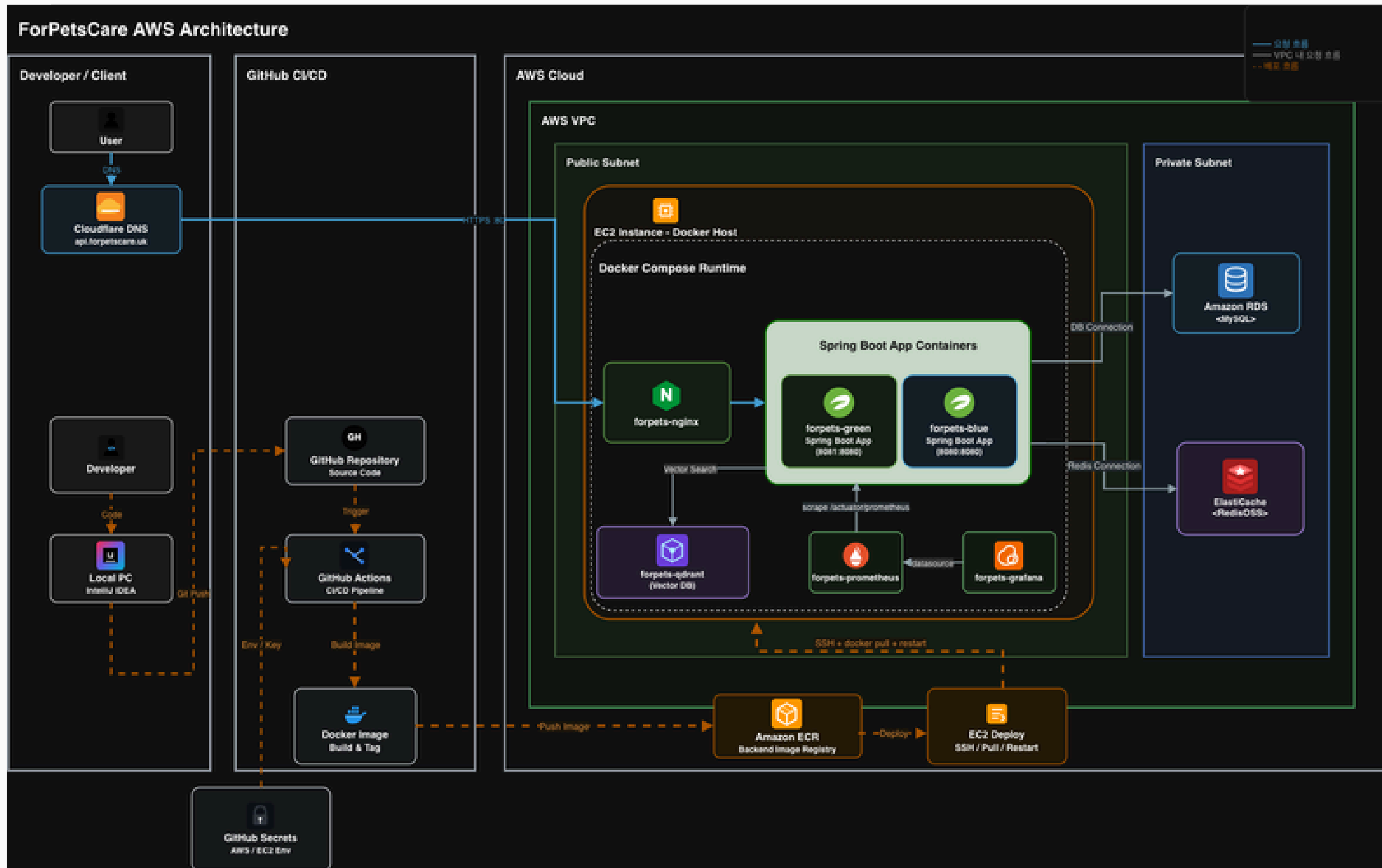
4. Release & Deploy
main 브랜치로 merge 후
배포



main 브랜치는 초기 세팅 및 배포 용도로만 사용
모든 기능 개발은 feature 브랜치 → develop 브랜치 PR Merge 방식으로 진행



AWS 운영 아키텍처



① 요청 진입

User → Cloudflare DNS → Nginx

② 애플리케이션 실행

EC2 내부 Docker Compose 기반
Spring Boot 컨테이너 운영

③ 데이터 계층

RDS(MySQL) / ElastiCache Redis를
Private Subnet에 분리

④ 배포 흐름

GitHub Actions → ECR → EC2 배포
자동화

✓ AWS 리소스 구성

i-0d44fa6d9fddb71 (forpets-app)에 대한 인스턴스 요약 정보

1 minute 전에 업데이트됨

인스턴스 ID
i-0d44fa6d9fddb71

IPv6 주소
-

호스트 이름 유형
IP 이름: ip-10-0-1-199.ap-northeast-2.compute.internal

프라이빗 리소스 DNS 이름 응답
-

자동 할당된 IP 주소
43.203.xxx.xxx [리플릭 IP]

IAM 역할
forpets-ec2-role

IMDSv2
Required

연산자
-

퍼블릭 IPv4 주소
43.203.xxx.xxx | [개방 주소법](#)

인스턴스 상태
실행 중

프라이빗 IP DNS 이름(IPv4만 해당)
ip-10-0-1-199.ap-northeast-2.compute.internal

인스턴스 유형
t3.small

VPC ID
vpc-03ea556e0690dfa98 (forpets-vpc)

서브넷 ID
subnet-0b2dd8252f1fa2506 (forpets-subnet-public1-ap-northeast-2a)

인스턴스 ARN
arn:aws:ec2:ap-northeast-2:367398821015:instance/i-0d44fa6d9fddb71

프라이빗 IPv4 주소
10.0.1.199

퍼블릭 DNS
ec2-43-203-xxx-xxx.ap-northeast-2.compute.amazonaws.com

탄력적 IP 주소
-

AWS Compute Optimizer 찾기
권장 사항을 위해 AWS Compute Optimizer에 옵티미합니다. | [자세히 알아보기](#)

Auto Scaling 그룹 이름
-

관리형
false

```

ec2-user@ip-10-0-1-199:~
forpets-green      367398821015.dkr.ecr.ap-northeast-2.amazonaws.com/forpets-backend:late
Up 2 days   0.0.0.0:8081->8080/tcp, :::8081->8080/tcp
forpets-grafana    grafana/grafana
Up 5 days   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp
forpets-qdrant     qdrant/qdrant:v1.9.7
Up 5 days   0.0.0.0:6333->6333/tcp, :::6333->6333/tcp, 6334/tcp
forpets-prometheus prom/prometheus
Up 5 days   0.0.0.0:9090->9090/tcp, :::9090->9090/tcp
[ec2-user@ip-10-0-1-199 ~]$

```

- EC2 t3.small 기반 운영 서버 구성
- Docker 컨테이너로 Nginx와 Spring Boot 앱 실행

- Nginx가 외부 요청을 app-blue / app-green으로 라우팅
- 단일 EC2 내부 멀티 인스턴스 구조 구성

✓ RDS / ElastiCache 구성

forpets-db

CPU 3.52%

클래스 db.t4g.micro

현재 활동 20 연결

리전 및 AZ ap-northeast-2c

연결 및 보안

모니터링

로그 및 이벤트

구성

제로 ETL 통합

유지 관리 및 백업

데이터 마이그레이션

태그

권장 사항

다음을 사용하여 연결 | 정보

코드 스니펫
SDK, API 또는 에이전트를 포함한 타사 도구를 통해 연결할 때 사용합니다.

CloudShell
AWS Management Console에서 직접 시작하는 AWS CLI에 빠르게 액세스하는 데 사용됩니다.

엔드포인트
모든 IDE 인터페이스를 통해 연결할 때 사용합니다.

데이터베이스 이름
forpets

마스터 사용자 이름
admin

인터넷 액세스 게이트웨이
비활성화됨

포트
3306

엔드포인트 유형
인스턴스 엔드포인트

추가 구성

연결 및 보안

엔드포인트 및 포트

네트워킹

보안

forpets-redis

클러스터 세부 정보

클러스터 이름
forpets-redis

설명
forpets-redis-dev

노드 유형
cache.t4g.micro

상태
Available

엔진
Redis

엔진 버전
7.1.0

글로벌 데이터 스토어
-

글로벌 데이터 스토어 역할
-

업데이트 상태
최신 상태

클러스터 모드
비활성화됨

사드
1

노드 수
1

다중 AZ
비활성화됨

자동 장애 조치
비활성화됨

전송 중 암호화
활성화됨

지장 중 암호화
활성화됨

파라미터 그룹
default.redis7

아웃포스트 ARN
-

전송 암호화 모드
읽기/기록

구성 엔드포인트
-

기본 엔드포인트
master.forpets-redis.bgk6tv.apn2.cache.amazonaws.com:6379

리더 엔드포인트
replica.forpets-redis.bgk6tv.apn2.cache.amazonaws.com:6379

ARN
arn:aws:elasticache:ap-northeast-2:367398821015:replicationgroup:forpets-redis

데이터 마이그레이션
활성 마이그레이션 없음

연결성 및 보안 - 신규

노드

지표

로그

유지 관리 및 백업

서비스 업데이트

태그

캐시에 연결 - 신규

AWS CloudShell, AWS CLI, GLUE 또는 기타 클라이언트 라이브러리를 사용하여 캐시에 연결합니다.

세부 정보

연결

네트워크 유형
IPv4

서브넷 그룹 이름
rds-private-subnet-group (vpc-03ea556e0690dfa98)

ARN
arn:aws:elasticache:ap-northeast-2:367398821015:private-subnet-group:rds-private-subnet-group

검색 IP 유형
IPv4

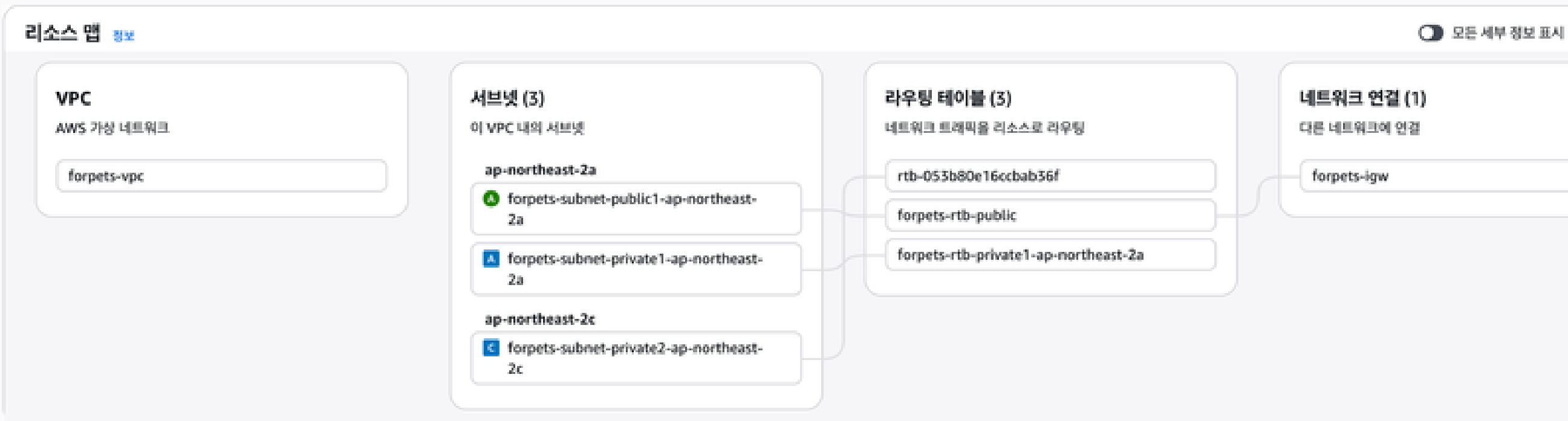
VPC ID
vpc-03ea556e0690dfa98

설명
rds-prd-private-subnet-group

- RDS MySQL 기반 핵심 데이터 저장소 구성
- Public Access 비활성화로 외부 직접 접근 제한

- ElastiCache Redis를 캐시 및 알림 이벤트 처리 인프라로 구성
- Redis Streams와 Pub/Sub을 활용한 비동기 알림 처리

✓ 네트워크 및 보안 구성



EC2 보안그룹

인바운드 규칙 (3) 태그 관리

🔍 검색

☐	Name	보안 그룹 규칙 ID	IP 버전	유형	프로토콜	포트 범위	소스
☐	-	sgr-0fe2c9d06a0c36909	IPv4	HTTP	TCP	80	0.0.0.0/0
☐	-	sgr-0f704f8ae2cca0430	IPv4	HTTPS	TCP	443	0.0.0.0/0
☐	-	sgr-0f11fe0148d4c8bad	IPv4	SSH	TCP	22	0.0.0.0/0

- EC2는 Public Subnet에 배치하여 외부 요청 진입점으로 구성

- RDS와 ElastiCache Redis는 외부 직접 접근을 제한

- Security Group을 통해 EC2에서만 DB/Redis 접근 허용

RDS 보안그룹

인바운드 규칙 (1) 태그 관리 인바운드 규칙 편집

🔍 검색

☐	Name	보안 그룹 규칙 ID	IP 버전	유형	프로토콜	포트 범위	소스	설명
☐	-	sgr-0f2da41b9c1164a83	-	MySQL/Aurora	TCP	3306	sg-09f7a07312c5b71a1...	-

- Nginx를 통해 외부 요청을 Spring Boot 컨테이너로 라우팅



검증 방향

단위테스트

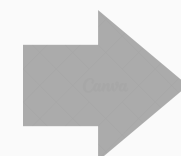
JUnit5로 단위·통합 테스트를 수행해
핵심 로직과 도메인 간 연동을 검증

통합·로컬 테스트

기능 단위 API 요청 흐름과
예외 응답을 시나리오별로 검증

API·E2E 테스트

배포 도메인에서 실제 사용자 흐름을
끝까지 실행하며 동작 여부 확인



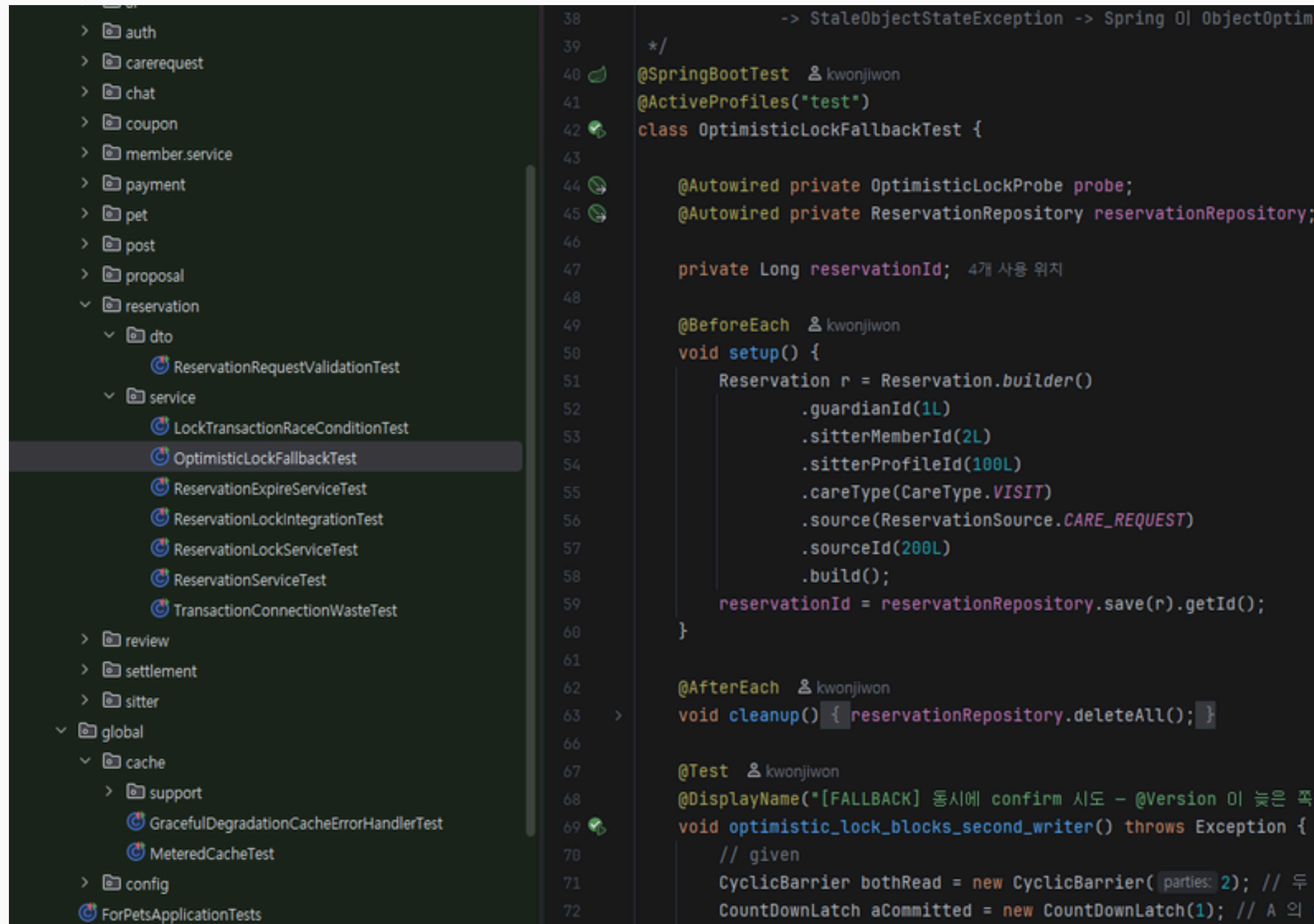
결과

로컬 개발 환경부터 배포 환경까지
단계별 테스트를 진행하여
기능 정합성과 서비스 안정성을
검증했습니다.



테스트 결과

IntelliJ 내부 단위테스트, 통합테스트



Postman 기능 단위 시나리오 테스트

A	B	C
번호	분류	테스트 케이스
1	회원/인증	[성공] 유효한 이메일, 비밀번호, 닉네임 입력 시 회원가입 성공 (기본 역할 MEMBER)
2	회원/인증	[실패] 이미 가입된 이메일로 회원가입 시도 시 중복 가입 차단
3	회원/인증	[실패] 이미 사용 중인 닉네임으로 회원가입 시 차단
4	회원/인증	[실패] 비밀번호가 형식 정책을 만족하지 않는 경우 가입 실패
5	회원/인증	[실패] 비밀번호가 형식 정책을 만족하지 않는 경우 가입 실패
6	회원/인증	[성공] 가입된 이메일과 올바른 비밀번호로 로그인 성공 — Access Token, Refresh Token 발급
7	회원/인증	[실패] 존재하지 않는 이메일로 로그인 시도 시 인증 실패
8	회원/인증	[실패] 비밀번호가 일치하지 않을 경우 로그인 실패
9	회원/인증	[실패] 정지된 계정으로 로그인 시도 시 차단
10	회원/인증	[성공] 유효한 Access Token으로 로그아웃 성공 — Redis 블랙리스트 등록
11	회원/인증	[실패] Authorization 헤더 없이 로그아웃 요청 시 차단
12	회원/인증	[실패] 만료된 Access Token으로 로그아웃 요청 시 차단
13	회원/인증	[성공] Refresh Token으로 Access Token 재발급 성공
14	회원/인증	[실패] 만료된 Refresh Token으로 재발급 시도 시 차단
15	회원/인증	[성공] 유효한 Access Token으로 내 정보 조회 성공
16	회원/인증	[성공] 닉네임, 전화번호, 성별 수정 성공
17	회원/인증	[성공] 현재 비밀번호 확인 후 새 비밀번호 변경 성공
18	회원/인증	[실패] 현재 비밀번호 불일치 시 변경 실패
19	회원/인증	[성공] 진행 중 예약 없을 때 회원 탈퇴 성공 — status DELETED 변경
20	회원/인증	[실패] 진행 중 예약이 있을 때 탈퇴 시도 시 차단
21	반려동물	[성공] 로그인된 회원이 반려동물 등록 성공 (name, species 필수)
22	반려동물	[실패] 반려동물 이름 누락 시 등록 실패
23	반려동물	[실패] 허용되지 않는 종 입력 시 등록 실패 (BIRD 등)
24	반려동물	[실패] 반려동물 10마리 초과 등록 시 차단
25	반려동물	[실패] 이름 50자 초과 등록
26	반려동물	[실패] 나이 음수 입력
27	반려동물	[실패] 나이 100 초과 입력
28	반려동물	[실패] 메모 2000자 초과 입력
29	반려동물	[성공] 내 반려동물 목록 조회 성공 — 본인 소유 목록만 반환
30	반려동물	[성공] 반려동물 상세 조회 성공
31	반려동물	[실패] 타인의 반려동물 조회 시 차단
32	반려동물	[실패] 존재하지 않는 반려동물 조회
33	반려동물	[성공] 반려동물 정보 수정 성공
34	반려동물	[실패] 타인의 반려동물 수정 시 차단
35	반려동물	[실패] 존재하지 않는 반려동물 수정
36	반려동물	[성공] 반려동물 삭제 성공 — 연관된 Post/CareRequest/Reservation 없는 경우
37	반려동물	[실패] 타인의 반려동물 삭제
38	반려동물	[실패] 존재하지 않는 반려동물 삭제
39	반려동물	[실패] 연관된 Post/CareRequest/Reservation이 존재하는 반려동물 삭제 시 차단
40	시터 프로필	[성공] MEMBER가 시터 프로필 등록 성공
41	시터 프로필	[실패] experienceYears 누락
42	시터 프로필	[실패] possiblePetType 누락
43	시터 프로필	[실패] possiblePetSize 누락
44	시터 프로필	[실패] pricePerHour 누락
45	시터 프로필	[실패] pricePerHour 0 입력
46	시터 프로필	[실패] pricePerHour 음수 입력

기능 소개

주요 기능

세부 기능

- 쿠폰 발급 동시성
- 성능 개선 (인덱싱/캐싱/결과)
- 알림 고도화
- AI

시연 영상

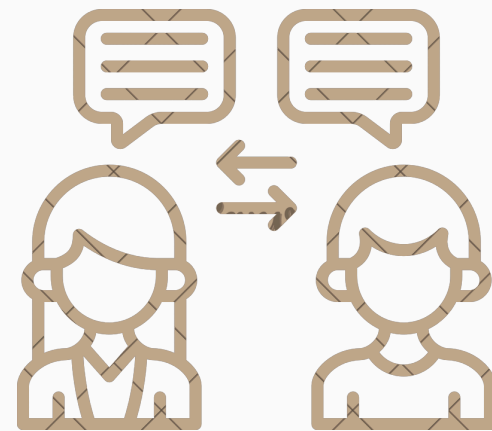
✓ 주요 기능 소개



반려동물 등록 & 공고 작성



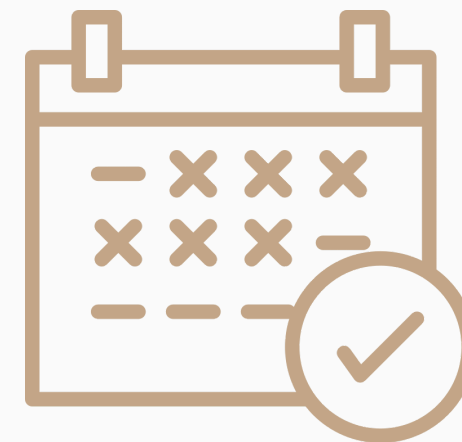
반려동물 정보 등록/관리
돌봄 요청 공고 작성
조건 기반 케어 요청



펫시터 매칭 & 서비스 관리



펫시터 프로필/서비스 등록
시터 스케줄 관리
순방향·역방향 매칭



예약 · 결제 · 정산 시스템



예약 생성 및 상태 관리
쿠폰 적용 및 결제 처리
예약 완료/취소 정산 처리



실시간 소통 & 리뷰 AI



실시간 채팅/알림
케어일지 작성
리뷰 요약 AI & 시터 추천 AI



쿠폰 발급 - 정합성

평가 기준: 어떤 Lock 이 초과 발급을 방지하는가?

시나리오 1

사용자 300명, 쿠폰 100개
(일반 경합)

시나리오 2

사용자 500명, 쿠폰 10 개
(극단적 경합)

Optimistic Lock 파라미터

- maxRetry: 10회
- backoffMs: 50ms

전략 \ 시나리오	S-1 (300명/100개)	S-2 (500명/10개)	정합성	소요시간
No 락	300 발급 (200 초과) ❌	324 발급 (314 초과) ❌	❌	1,630ms █████
낙관적 락	92 발급 (8 부족) ⚠️	10 발급	⚠️	4,085ms ████████████████████
비관적 락	100 발급	10 발급	✅	2,050ms ██████████
분산 락	100 발급	10 발급	✅	2,420ms ████████████████



쿠폰 발급 - 채택: Redis 분산락

Redis 분산락 채택 이유

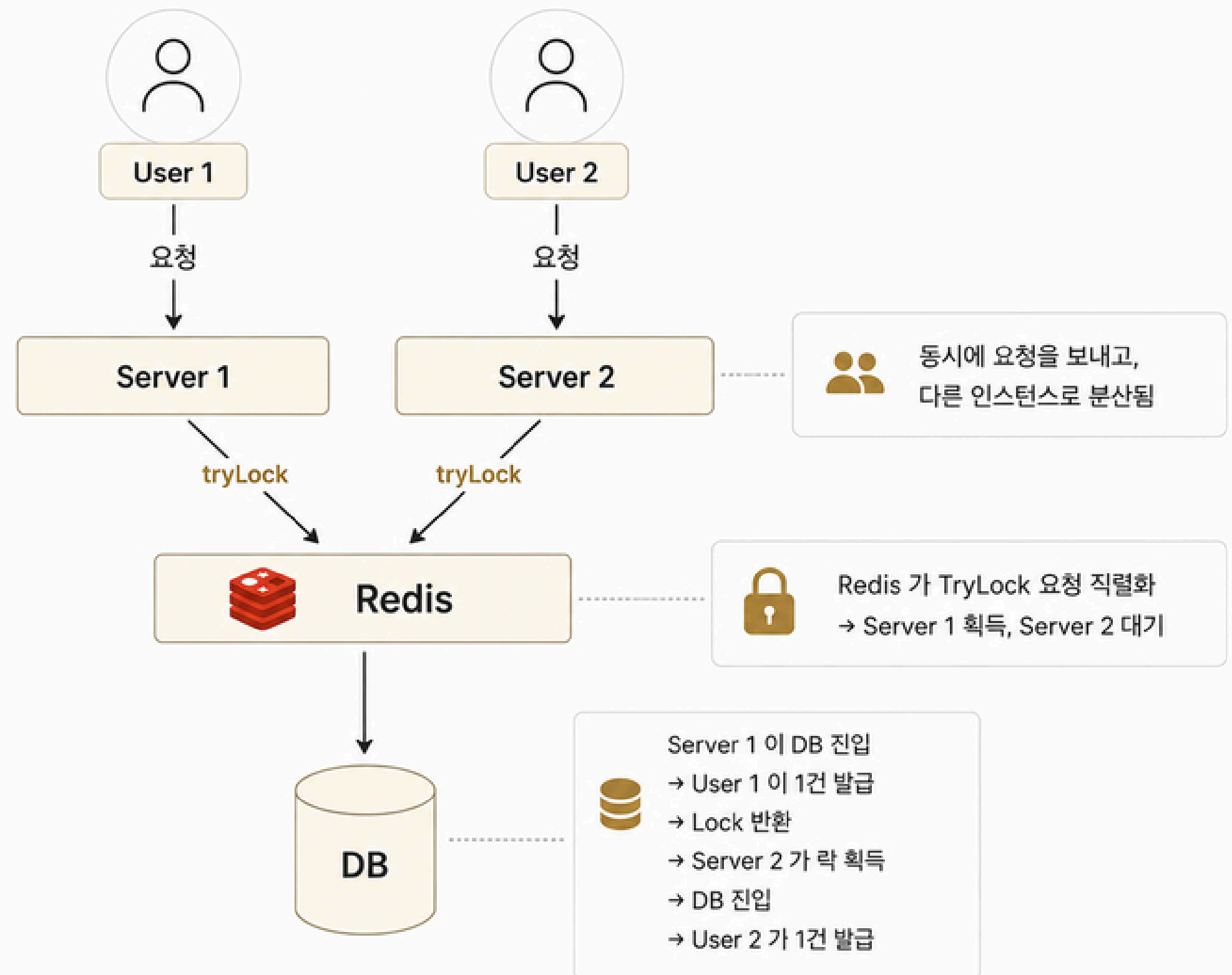
- 1. 다중 인스턴스 환경 대응 가능
- 2. DB 자원 보호
- 3. 인프라 추가 비용 없음

Redis 세팅

락 키: lock:coupon:issue:{couponId}

락 단위: 쿠폰 ID 단위 (도메인 영향 최소화)

실패 응답: 409 COUPON_ISSUE_LOCK_FAILED





쿠폰 발급 - 임계값

임계값 모니터링 테스트

	Pessimistic (비관락)			Distributed (분산락)		
VU	성공	HikariCP pending	500 오류	성공	pending	500 오류
100	100	0	0	100	0	0
300	300	0	0	300	0	0
325	—	—	—	325 ✓	0 (875ms 대기)	0
350	350	119 ⚠️ (2.34s 대기)	0	340	0	0
400	291 ✖️	165	109 ✖️	87	0	0

한계점과 개선방향

Redis 단일 인스턴스 의존

Redis 장애 시 락 자체 동작 불가

개선: Redis Sentinel 검토
RedLock과 트레이드 오프 고려

waitTime 3초

사용자 체감 시간

개선: 큐 기반 비동기 발급
(Kafka) 으로 전환 검토

락 실패(409)의 클라이언트 처리 정책

자동 재시도 정책

개선: 재시도 정책과 더불어
대기열 UI 도입



성능개선 개요

성능 개선 대상 쿼리

GET /api/sitters
GET /api/sitters/{id}
GET /api/posts

구조

동적 where, ordering, Pagination

검색 조건

region · gender · petType ·
petSize · minPrice · maxPrice

더미 데이터 구성

규모

member 10만 + sitter_profile 10만

현실과 유사한 **데이터 분포** 구성

ex) 이용가능 85%, 이용 불가능 15%
승인 75%, 대기 15%, 거절 10% 등

재현성

Random seed 고정

테스트 설정

도구

k6

시나리오

constant-vus: 50 VU * 3m

요청 간격: 100ms

목표 요청 수 30,000 +

p95 < 200ms · p99 < 200ms

error rate < 1%

랜덤으로 검색 조건을 선택하여 요청
→ 실 트래픽 모사

✓ 성능개선 — Index, Cache

Index 도입 이유

동적 조건 쿼리 검색
조건 조합별 인덱스 채택 상이
Member table 과 JOIN 존재

➡ **쿼리를 빠르게**

- (possible_pet_type, created_at) - sitter_profile
- (possible_pet_type, created_at, possible_pet_size, approval_status) - sitter_profile
- (region, gender) - member

선정 기준

등치 조건, 정렬(created_at) 컬럼 우선
Cardinality 고려
EXPLAIN 분석 (eq_ref 사용: **100,003 건 → 4,004 건**)

Cache 도입 이유

시터 프로필: read-heavy 도메인
동일 검색 조건 반복 요청 가능

➡ **DB 를 쉬게**

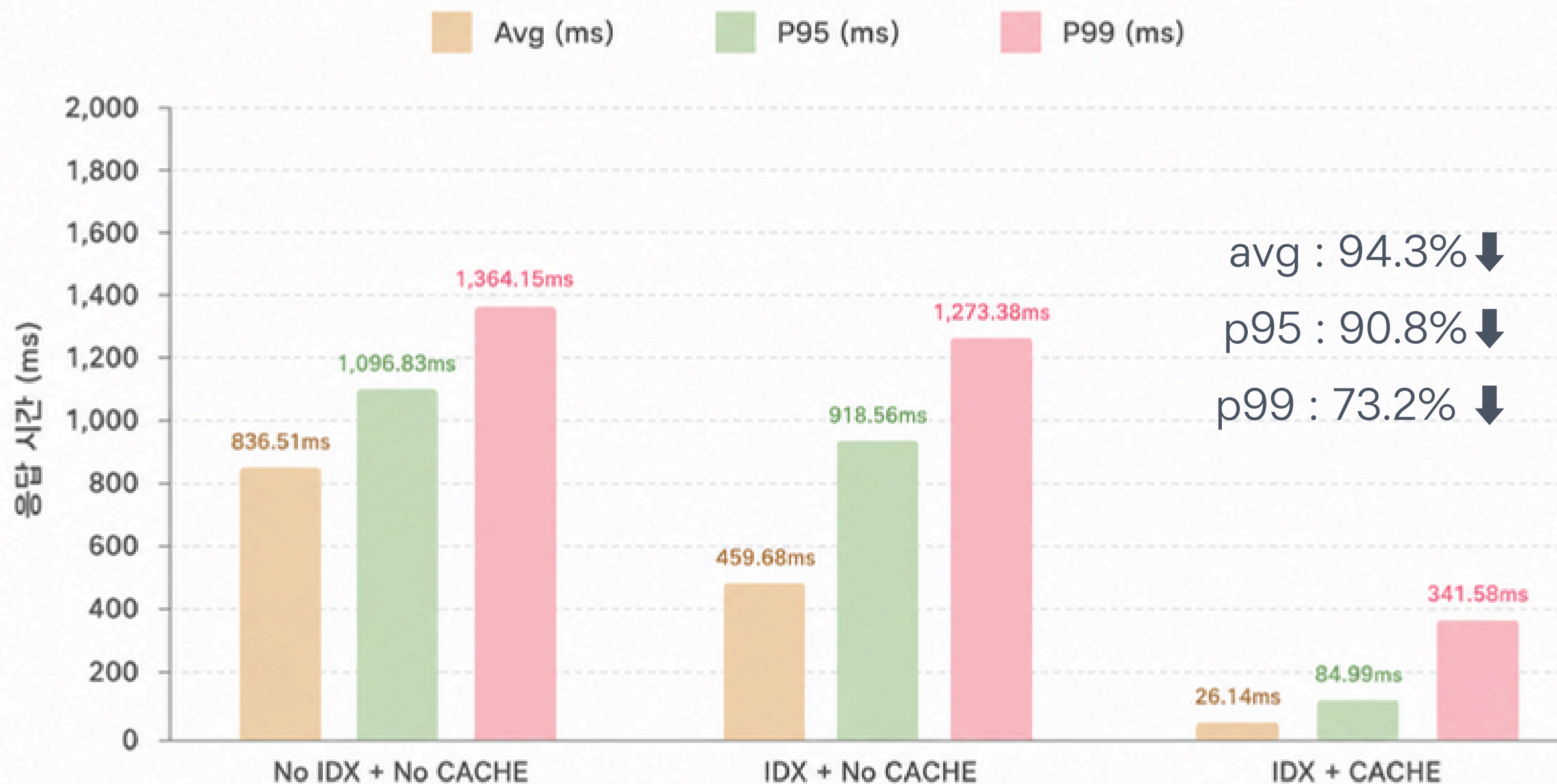
	longTtlCacheManager	shortTtlCacheManager
TTL	1 H	5 m
대상	sitters 목록 sitter 단건	postings 목록

캐시 키 설계 : cache:sitters:{sha-256 hash}

10개 파라미터 해시(검색조건+페이징·정렬) : SHA 256
→ Cache key 길이 일정, 유일성 보장

✓ 성능개선 — 결과

서버 목록 조회 성능 비교



추가 지표

캐시 적중률 **88.5 %**

(전체 요청: 46,554개)

RPS **7.38배**

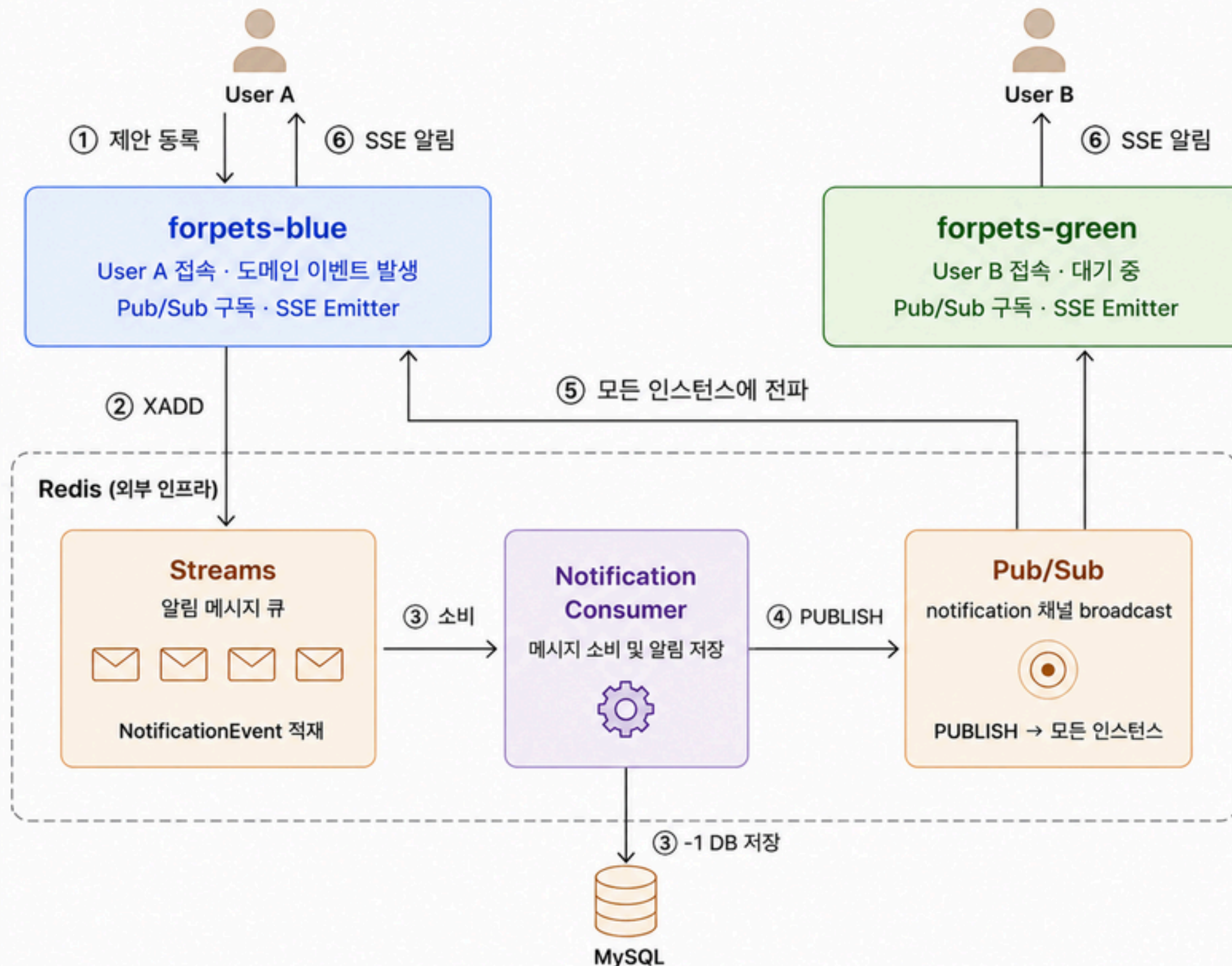
(no IDX + no CACHE vs IDX + CACHE)

53.11 → 392.07

개선 방향

- 1 운영 전 캐시 워밍업 전략
- 2 캐시 무효화 범위 최적화

실시간 알림 시스템 고도화



- ① 도메인 이벤트 발생
- ② Redis Streams 발행
- ③ Consumer 메시지 소비 및 DB 저장
- ④ Redis Pub/Sub 발행
- ⑤ 모든 앱 인스턴스에 전파
- ⑥ 연결된 인스턴스에서 SSE 전송

NOTIFICATIONS

알림

요청, 제안, 결제처럼 바로 확인해야 하는 이벤트를 모아 봅니다.

안 읽은 알림

미읽음 1건

예약 완료 미읽음

관련 화면

읽음 처리

예약 결제 제한 시간이 초과되어 예약이 만료되었습니다.

2026. 6. 19. 오후 11:22

결제 완료 읽음

관련 화면

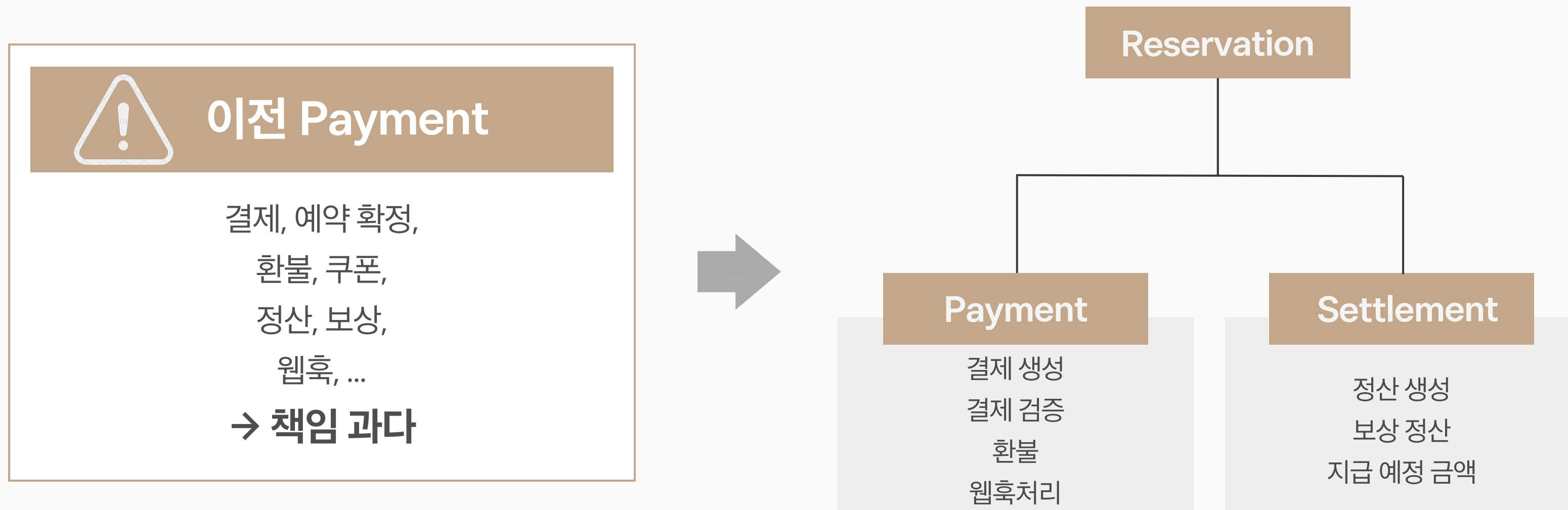
읽음 완료

결제가 완료되어 예약이 확정되었습니다.

2026. 6. 19. 오후 9:41

✓ DB 저장 후 Pub/Sub 발행
실시간 전송 실패 시에도 알림 이력 보존

✓ Payment와 Settlement를 분리한 이유



**“플랫폼이 돈을 받는 영역”과 “플랫폼이 돈을 지급하는 영역”을 분리하여
도메인 책임을 명확하게 설계**

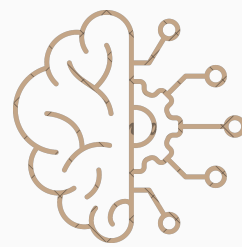
✓ AI 기능 소개

✓ 단순 조건 매칭을 넘어, 시터 정보와 실제 보호자 리뷰 근거를 함께 활용해 맞춤형 시터 추천과 리뷰 요약 제공



사용자 질문 입력

지역, 일정,
반려동물 성향 등
원하는 돌봄 조건 입력



AI 조건 분석

사용자 요청에서
추천 기준과
핵심 조건 추출



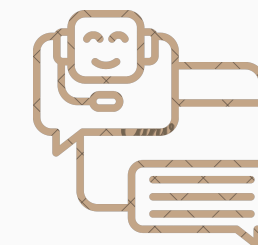
시터 후보 조회

지역, 일정,
서비스 조건에 맞는
펫시터 후보 조회



RAG 리뷰 검색

Qdrant에서
실제 리뷰 기반
유사 근거 데이터 검색



AI 응답 생성

Gemini가
시터 정보와
리뷰 근거를 바탕으로
추천 생성



추천 결과 제공

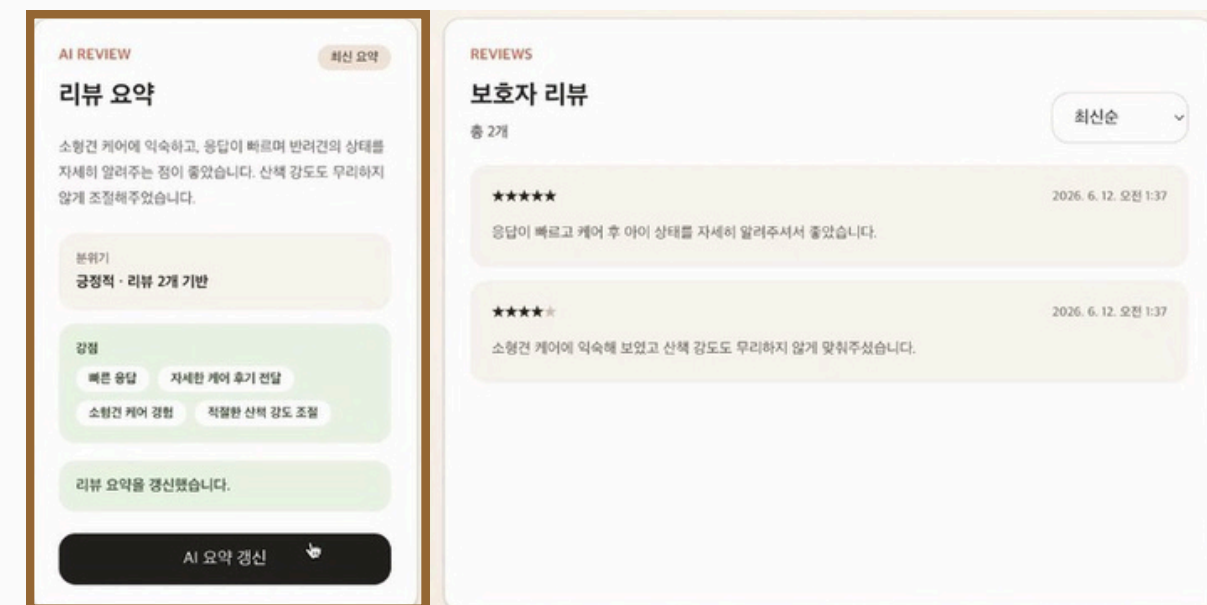
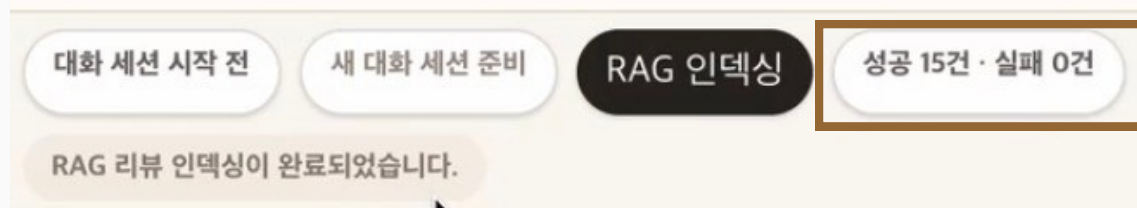
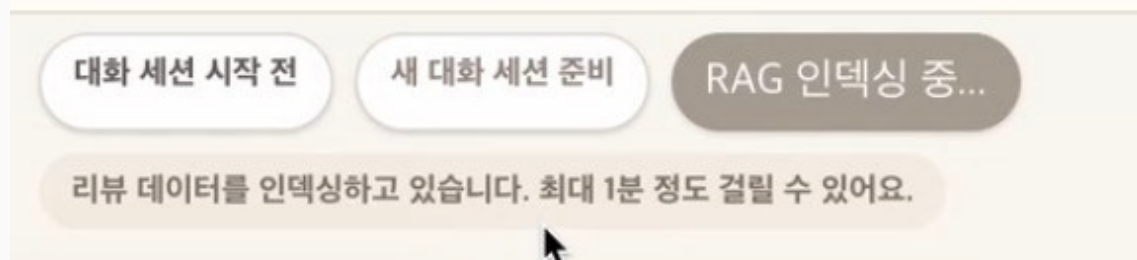
추천 시터 카드와
리뷰 근거를
함께 표시



AI 기능 구현 디테일

RAG 검색, 배포 환경, 인덱싱 관리까지 고려한 AI 기능 안정화

관리자 계정으로 접속 시 인덱싱 가능



RAG 인덱싱 관리

리뷰 데이터를 Qdrant에 적재해
AI 추천과 리뷰 요약의 근거로 활용

→ 관리자 전용 인덱싱 기능을 두고
성공/실패 건수를 분리해 추적

배포 환경 RAG 안정화

로컬에서는 정상 동작했지만
배포 환경에서 Qdrant URL 설정 누락으로
RAG 검색 실패 발생

→ 환경 변수와 Docker 네트워크 설정을 분리해
배포 환경에서도 검색 가능하도록 수정

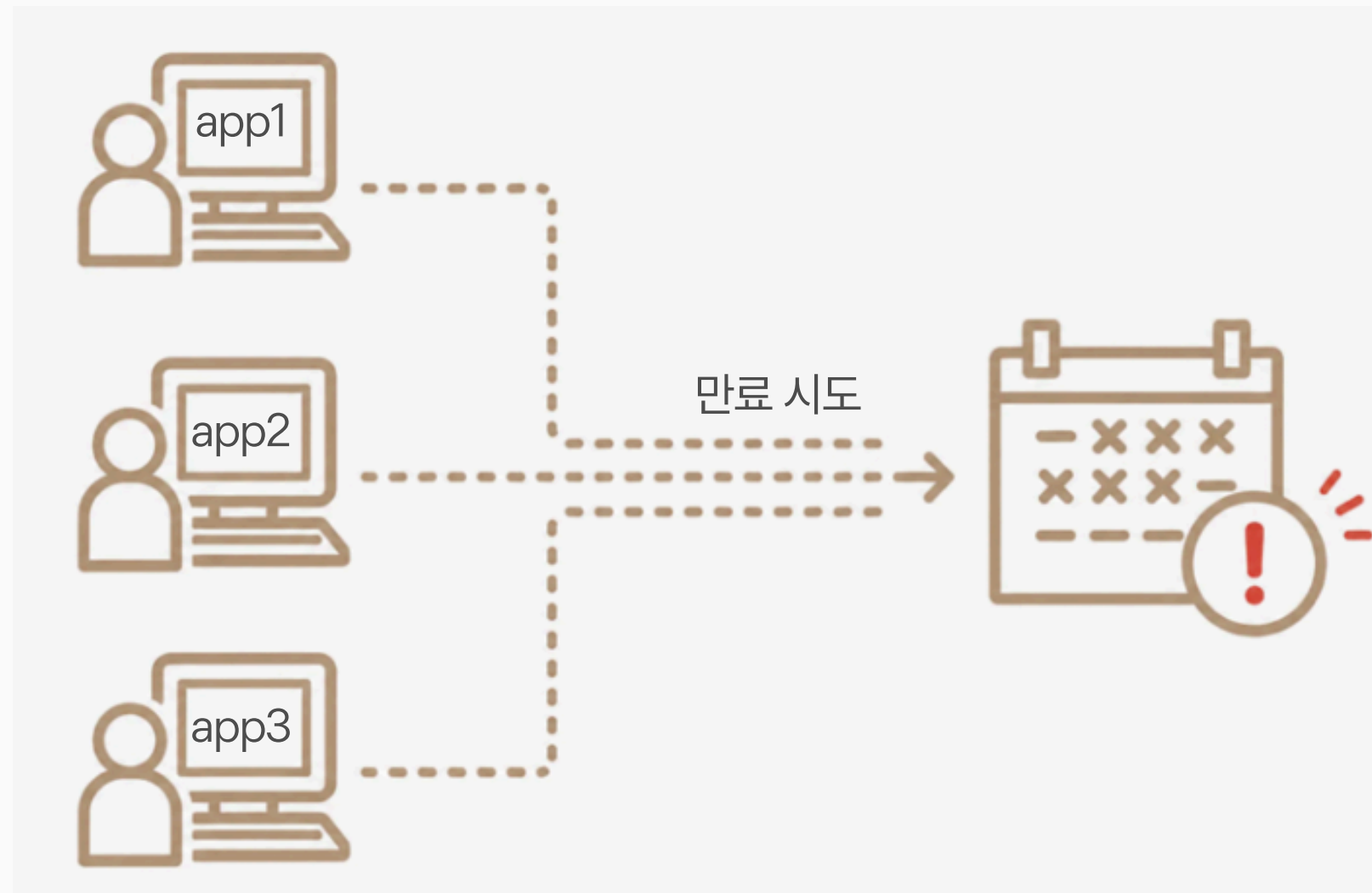
프롬프트 템플릿 관리

배포 환경에서 활성화된 프롬프트 템플릿이
없어 AI 리뷰 요약 생성 실패

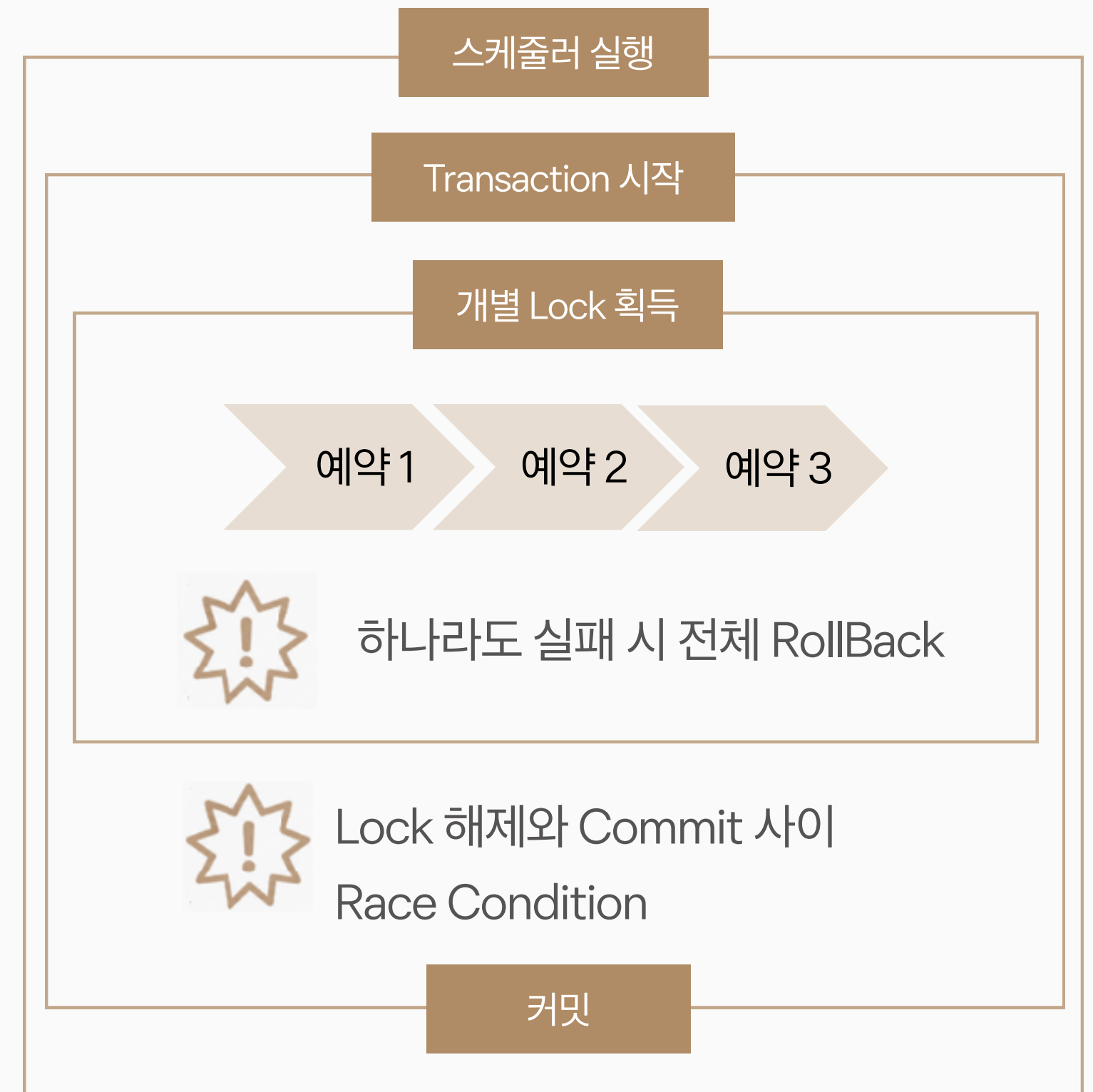
→ 환경별 프롬프트 로딩 구조를 점검하고
배포 환경에서도 리뷰 요약이 생성되도록 수정

✓ 다중 인스턴스 환경 + 동시성 문제

다중 인스턴스 환경 (스케줄러)



!
 자원 하나에 여러 요청 발생 시
 단일 인스턴스 Lock 으로는 대응 불가





리팩토링, 2중 락, ShedLock

리팩토링

목록 단위 Tx → 단건 단위 Tx
부분 실패 격리

스케줄러 실행

개별 Transaction + Lock

예약 1

개별 Transaction + Lock

예약 2

개별 Transaction + Lock

예약 3

분산락

기본 동시 접근 차단

① Lock Service 분리
→ Lock 위치가 TX 외부

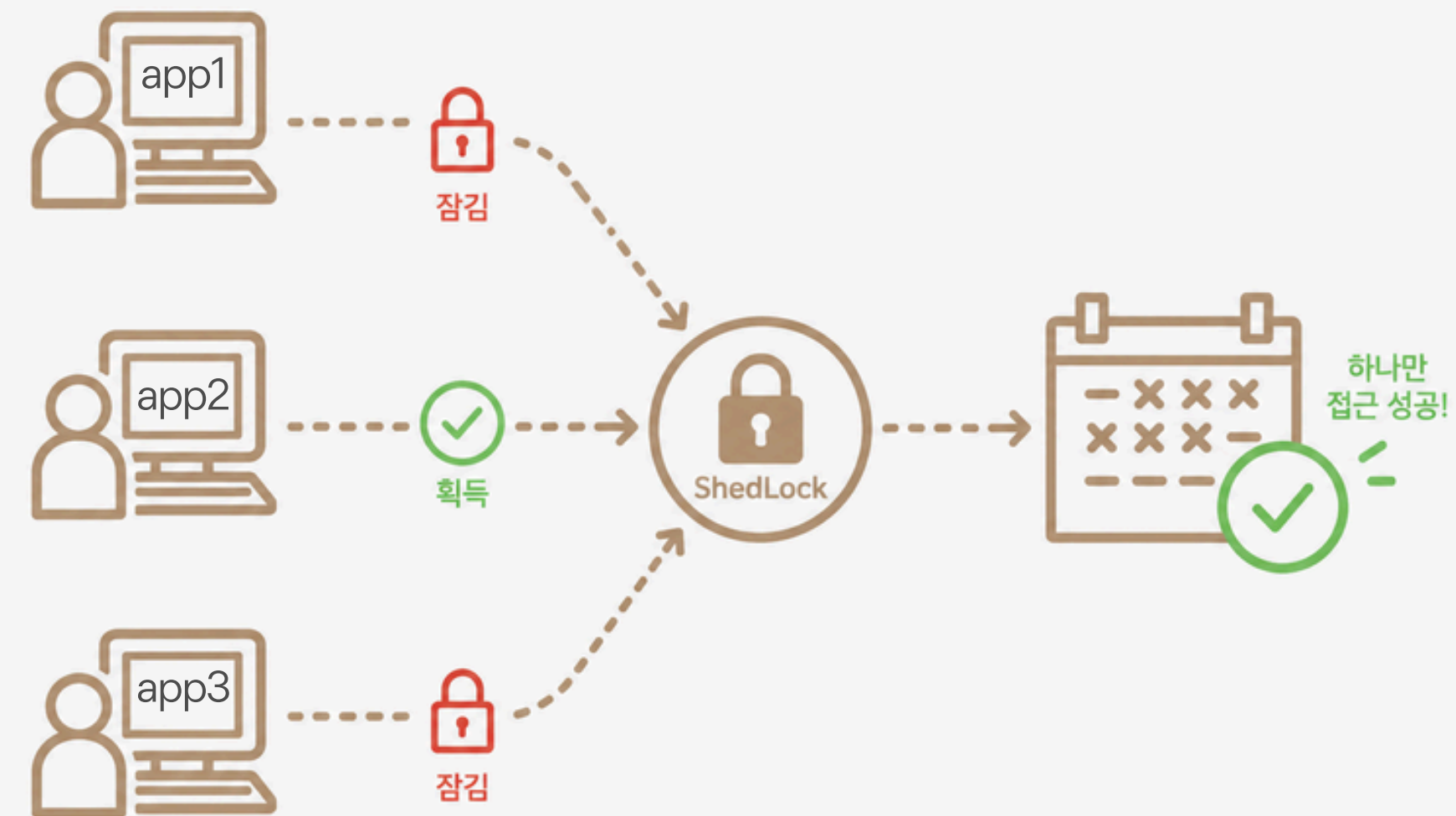
② DistributedLock AOP
→ 재사용성, 가독성 향상

낙관락

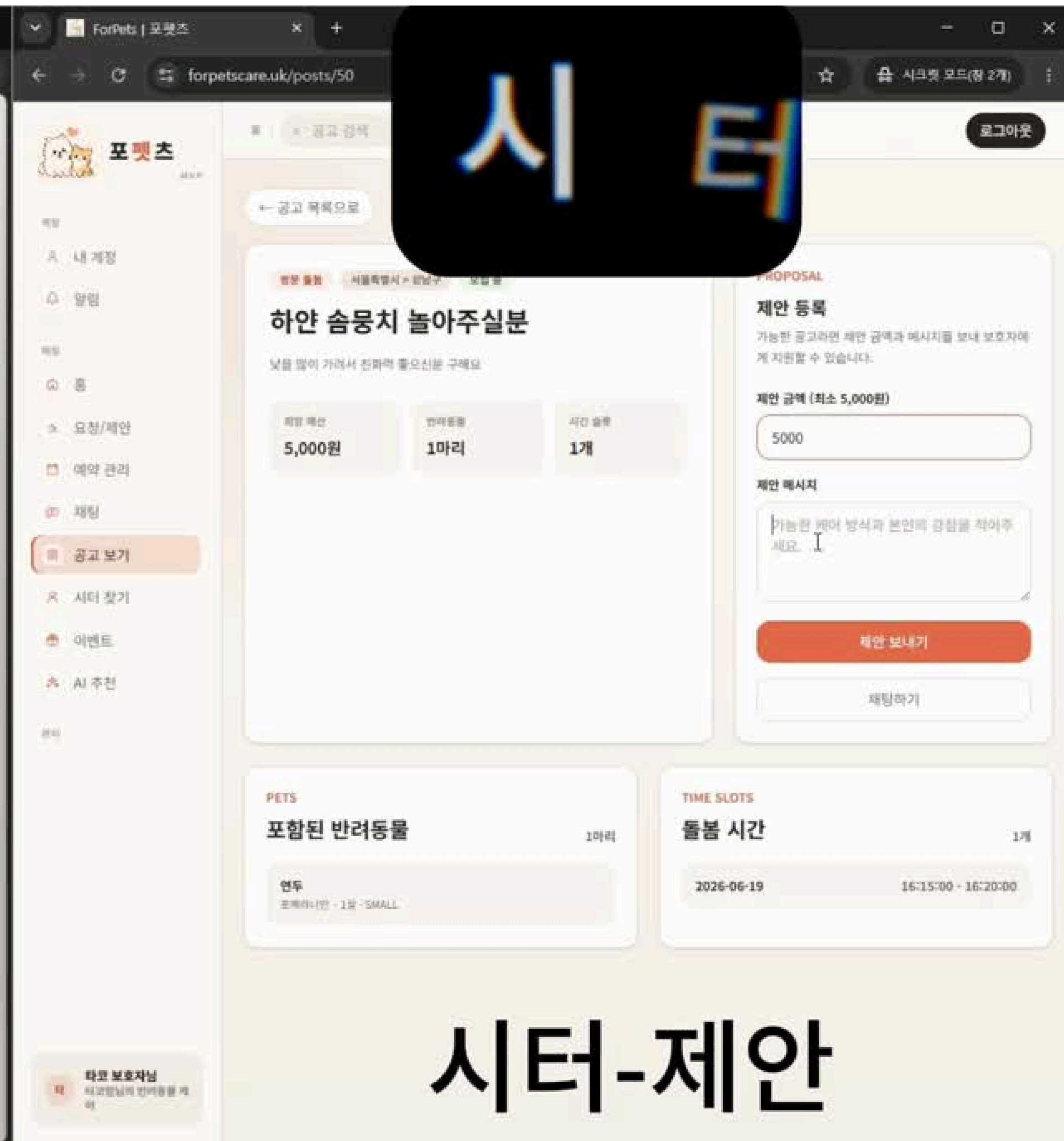
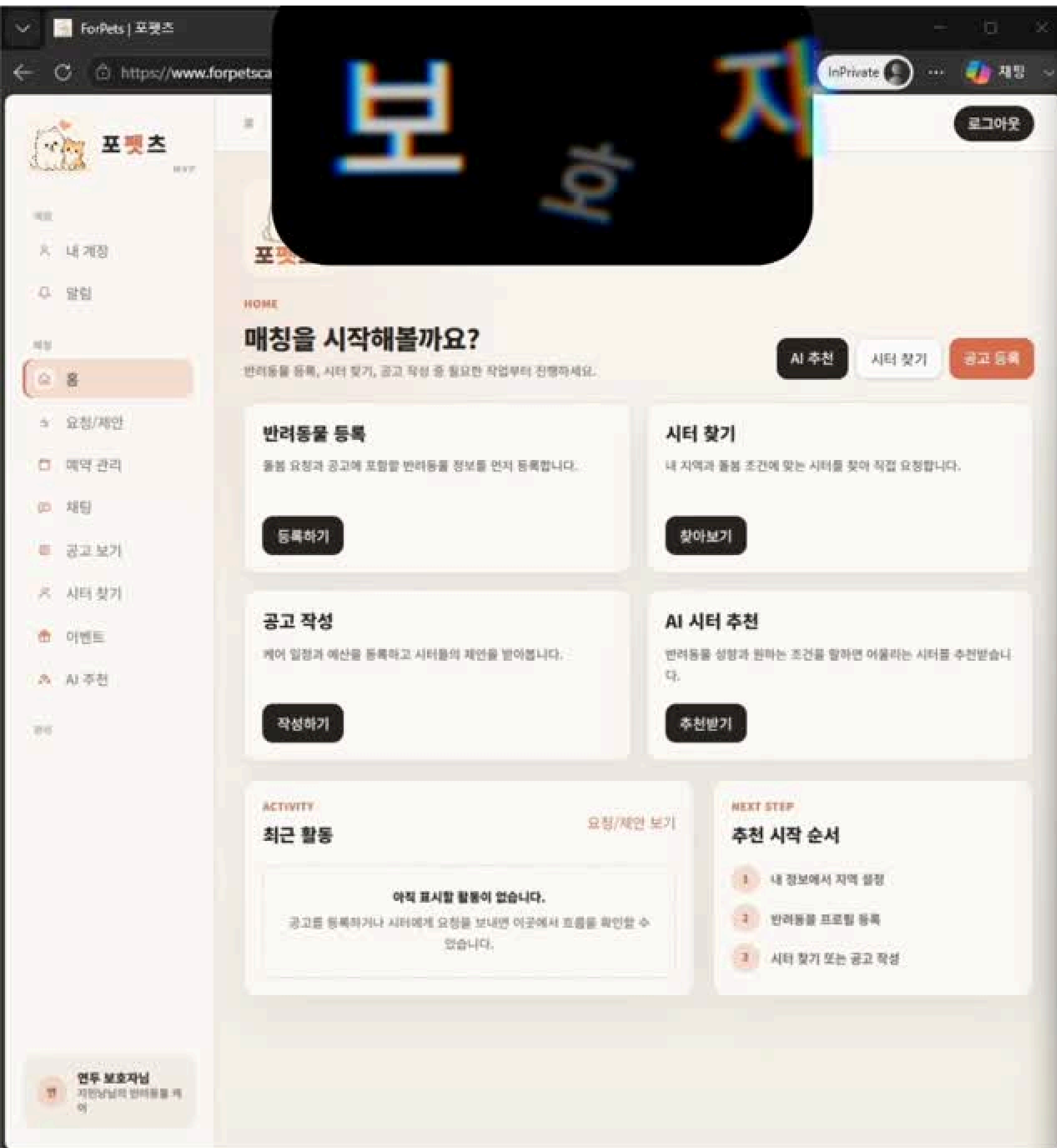
Redis 장애 시에도
Race Condition 방지
→ 이중 방어 구조

ShedLock

스케줄러 전용
중복 실행 방지



시연 영상



시터-제안

기술 고도화 및 서비스 확장 계획



인기 시터 TOP10

예약·리뷰 데이터를 점수화하고
 Redis ZSet을 활용해
 실시간 인기 시터 랭킹 제공



부분 환불·정산 고도화

취소 정책과 PG 정책을
 반영한 부분 환불 및
 정산 자동화 로직 개선



위치 기반 시터 검증

외부 신원 인증 API와
 GPS 인증을 연동해 신뢰도
 높은 시터 검증 체계 구축



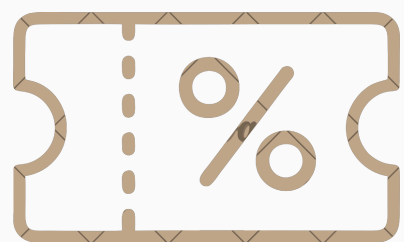
반려동물 커뮤니티

반려생활 커뮤니티를 확장하고
 이미지 유사도 검색으로
 보호자 알림 기능 제공

기존 기능의 성능과 안정성을 고도화하고,
 신뢰 기반 기능과 커뮤니티 확장을 통해 반려생활 전반을 지원하는 플랫폼으로 발전할 계획

트러블슈팅

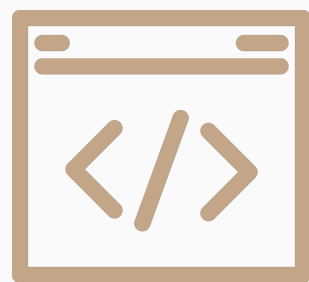
✓ 주요 트러블슈팅



쿠폰 임계값 테스트 측정 기준 재설계



응답값과 락 경합 원인 분리 및
정확한 성능 측정 기준 설정



Spring AOP 체이닝 AOP 바인딩 구조 개선



Advice 중첩 원인 분석 및
파라미터 바인딩 방식 개선



Redis Streams 알림 처리 Consumer Group 초기화



SSE 알림 누락 원인 분석 및
메시지 처리 구조 개선



Graceful Degradation 장애 대응 구조 설계



Redis 장애 원인 분석 및
DB 조회 Fallback 처리 적용



AI/RAG 배포 환경 Qdrant 연결 설정 개선



RAG 검색 실패 원인 분석 및
배포 환경 최적화

✓ 쿠폰 임계값 테스트 설계 문제

✓ 문제상황

임계값 측정 중 VU 1,000까지 증가해도
정합성 유지되는 현상 발생

원인: 여러 응답이 섞여서 출력
발급 성공 201
쿠폰 소진 400
락 경합 409

쿠폰 소진 시 락 경합 없이 즉시 응답
VU 증가가 부하로 이어지지 않음

✓ 해결과정

쿠폰 수량 $\geq$ VU 로 재설정
→ 모든 요청이 발급 로직까지 진입

: 쿠폰 소진 변수 제거
응답시간 영향 변수를
락 경합 단일 요인으로 한정

: 임계값 재정의
avg·p95 응답시간의 기울기가
급격히 꺾이는 지점

✓ 회고

부하 테스트 설계 자체가
측정 결과를 왜곡할 수 있음

테스트 시나리오에서
"측정 대상 외 변수"를 제거하는 것이
정확한 임계값 산출의 전제 조건



Spring AOP 체이닝

✓ 문제상황

@DistributedLock

@TrackExecutionTime

두 AOP를 같은 메서드에 적용

→ IllegalStateException 발생

매칭 + 파라미터 바인딩을 동시 요청

: 두 번째 Advice 진입 시점에

JoinPointMatch 매핑 정보 누락

→ 바인딩 실패

✓ 해결과정

1. argNames 직접 지정 시도

→ 근본 원인은 체이닝 중 매핑 누락

→ 동일 오류 발생, **실패**

2. 바인딩 자체 제거

@annotation(풀 클래스명) 형태 채택

→ Spring에는 매칭만 요청

+ **Reflection**으로 직접 조회

→ 런타임에 어노테이션 인스턴스 추출

✓ 결론

JoinPointMatch 바인딩 의존 제거

→ 체이닝 시 누락 이슈 원천 차단

@annotation() → 매칭 + 바인딩

@annotation(클래스명) → 매칭만

Advice 2개 이상 체이닝 시

Reflection 직접 조회 방식이 안전

✓ Graceful Degradation 트러블슈팅

✓ 문제상황

Spring Cache는
**Redis 장애 시 예외를
그대로 던져**, 캐시와 무관한 조회 API
까지 500에러가 발생함.

✓ 해결과정

CacheErrorHandler를 구현한
**GracefulDegradationCacheErrorHan
dler**로 캐시의 기능인
GET / PUT / EVICT / CLEAR
실패를 log.warn으로 삼키고 **DB 직접
조회로 폴백**
로그에 [CACHE_DEGRADED] 키워드를
상수로 박아 필터링·알람 연동을 쉽게 했다.

✓ 배운 점

캐시는 성능 수단이지 필수 인프라
가 아니다.
**Redis가 죽으면 서비스는 느려질
수는 있어도 멈추면 안 된다.**
장애 시 동작을 명시적으로 설계해
야 "캐시 때문에 전체가 죽는" 사고
를 막는다.

✓ Redis Streams Consumer Group

✓ 문제상황

Redis Stream에는 메시지가 쌓였지만, Consumer Group에 **Consumer가 등록되지 않음**

=> 그 결과 메시지가 소비되지 않아 DB 저장과 SSE 알림 전송이 **누락**

✓ 해결과정

1. **Consumer 시작 시점을 SmartLifecycle**로 분리하여 애플리케이션 시작 시 반드시 초기화 하도록 수정

2. **MKSTREAM**으로 Stream이 없을 때도 Group을 생성
BUSYGROUP은 재시작 상황으로 예외 처리

✓ 배운 점

Redis Stream에 메시지가 쌓였다고 전체 알림 흐름이 정상인 것은 아니었음

비동기 메시지 구조는
발행 → 저장 → 소비 → DB 반영 → 실시간 전송을 단계별로 나누어 확인해야함

✓ 배포 환경에서 RAG 검색이 실패한 문제 해결

✓ 문제상황

로컬에서는 AI 시터 추천과
리뷰 근거 표시가 정상 동작

하지만 배포 환경에서는
AI 요청 시 network error 발생

서버 로그 확인 결과
Qdrant 요청이 **localhost:6333**
으로 나가며
Connection refused 발생

✓ 해결과정

Spring Boot 컨테이너 내부의
localhost는
Qdrant가 아닌 자기 자신을 의미

배포 환경에서는 Docker Compose
서비스명인

http://qdrant:6333을 사용하도록 수정

QDRANT_BASE_URL 등록
SPRING_PROFILES_ACTIVE=prod,ai
적용

✓ 회고

AI/RAG 기능은 코드 배포만으로
끝나지 않고
Vector DB, 외부 API Key, Profile,
컨테이너 네트워크 설정까지
함께 맞춰야 함

로컬 성공과 배포 성공은
다르기 때문에
배포 환경 기준 체크리스트의
필요성을 느낌

✓ 캐시 hit/miss 측정 트러블슈팅

@Cacheable 프록시 구조 한계를 MeteredCacheManager로 해결

✓ 문제상황 · 원인

캐시 적중률을 재려고 서비스 메서드에 카운터를 넣었더니 miss만 세어지고 hit은 계속 0이었다.

원인 (두 가지가 겹침)

- ① @Cacheable은 미스일 때만 메서드가 실행됨 → 히트는 프록시가 가로채 hit 카운트 누락
- ② Micrometer 자동 계측은 RedisCache 미지원

✓ 해결과정

hit·miss가 동시에 보이는 Cache.get()을 데코레이터(MeteredCache)로 감싸고, CacheManager 레벨에서 통째로 래핑(MeteredCacheManager). 이후 캐시가 추가돼도 자동 계측되며, 태그는 캐시명만 달아 카디널리티 폭발을 막았다.

✓ 배운 점

"측정을 어디에 넣을지"는 기능 위치가 아니라 프레임워크가 어느 레이어(프록시/추상화)에서 동작하는지를 따라가야 한다. @Cacheable이 메서드 바깥에서 도는 걸 모르면 영원히 절반만 센다.

✓ 운영 데이터 분포 반영 트러블슈팅

CLOSED 누적 분포로 데이터를 반영하니 인덱스 설계가 통째로 뒤집힘

✓ 문제상황

광고 검색 인덱스를 처음엔 더미데이터 OPEN 70% / CLOSED 30% 분포 위에서 설계했다.

그런데 실서비스는 정반대다 — 광고는 시간이 지나면 마감/만료되어 CLOSED가 누적되고 OPEN은 소수가 된다(성숙한 서비스면 OPEN 5~15% 수준). 비현실적 분포 위에서 내린 인덱스 결정은 운영에서 틀린다.

✓ 해결과정

운영 비율로 재삽입 후 재측정함. 같은 인덱스 2개를 둔 채 같은 쿼리로 EXPLAIN을 다시 시도해보자 옵티마이저가 선택을 뒤집었다.

Before: OPEN 70% (비현실) →
선택된 인덱스 : idx_post_deleted_created (2컬럼)

After: OPEN 10% (운영 분포) →
선택된 인덱스 :
idx_post_deleted_status_created (3컬럼)

✓ 배운 점

더미데이터는 양보다 분포(선택도)의 현실성이 중요하다.

분포 하나로 옵티마이저의 인덱스 선택과 설계 결론이 뒤집혔다 — 인덱스 설계는 운영 분포를 재현한 데이터 위에서 해야 한다.

Race condition 문제

문제상황

스케줄러에 @Transactional 적용
→ 만료 대상 전체 단일 Tx 내부 처리

목록이 많을수록 Tx 유지 기간 증가

다른 예약에 대한 요청이 들어오면
Lock 경합 발생, 트랜잭션 전체 실패

Lock 이 @Transaction 내부에 위치
→ Lock 해제, 커밋 사이 빈 시점 존재
→ Race Condition 발생

해결과정

1. 스케줄러의 Tx 범위 변경
목록 단위 → 단건 예약 단위
: 1건 실패가 전체에 영향 X

2. Tx 밖으로 Lock 위치 이동,
@DistributedLock AOP 적용
: Lock 과 커밋 사이 순서 보장

3. 낙관적 락 (@Version) 추가
: Lock 이 놓친 미세 Race 도 방지

결론

스케줄러 단건 처리로 부분 실패
격리, 전체 안정성 확보

Lock 획득
→ Tx 시작
→ 비즈니스 로직
→ Commit
→ Lock 해제

분산락, 낙관적 락 2중 방어



회고

- ✓ **최길중** 쿠폰 동시성 제어를 구현하며 락 전략마다 장애 패턴과 DB 커넥션 영향이 달라진다는 걸 직접 체감했고, 기술 선택이 단순한 구현 문제가 아니라 정책과 운영 안정성까지 고려해야 하는 설계 문제임을 배웠습니다.
- ✓ **권지원** 메인 비즈니스 로직을 구현하며 스케줄링, 동시성, 모니터링을 편하게 넣을 수 있는 구조를 생각했습니다. 단지 기능을 동작하게 하는 것이 아닌 어떻게 구조를 짜고 기획을 해야하는가를 직접 설명할 수 있게 된 부분에서 마지막 프로젝트를 잘 마무리 한 것 같습니다!
- ✓ **박영수** Redis Streams, Pub/Sub, SSE, EC2 멀티 인스턴스 구조를 적용하면서 알림이 안정적으로 전달되기 위해 어떤 흐름이 필요한지 배웠습니다. 단순히 "돌아가는 코드"를 넘어서 배포, 리소스, 네트워크까지 운영에서 필요한 것들까지 함께 생각하는 백엔드 경험을 얻어서 좋았습니다.
- ✓ **선경안** 성능개선(캐싱/인덱싱) 부분을 진행하며 구현된 기능들이나 DB조회 속도 등을 분석하고 이를 토대로 개선해가는 과정이 있었습니다. 성능개선시 실제 운영쪽 데이터나 상황을 고려하여 진행해야 결과를 신뢰할 수 있다는 점을 배웠습니다. 조회나 리뷰 기능을 포함해 초반 정책설계나 ADR을 위한 학습 등 과정이 쉽지는 않았지만 하나씩 알아가는 기쁨을 경험할 수 있어 좋았습니다.
- ✓ **이지민** 역할은 나누었지만 예약, 결제, 알림, AI 기능이 서로 연결되어 있었기 때문에 API 명세와 정책을 지속적으로 맞추는 과정이 중요했습니다. 이번 프로젝트를 통해 단순 기능 구현을 넘어 도메인 간 연결, 상태 전이, 배포 환경, 장애 대응까지 고려하는 백엔드 설계의 중요성을 느꼈습니다.

Q&A

경청해주셔서 감사합니다.

집사조일체

최길중 | 권지원 | 박영수 | 선경안 | 이지민

배포

<https://forpetscare.uk/>



스파르타 내일배움캠프
최종 프로젝트

GitHub

<https://github.com/Housekeeper-for-pets>